

**CAMPUS
DIGITAL FP**

ShinyVillage – Videojuego casual desarrollado en Unity

Cristina García Quintero

Tutor: Alejandro Torres Del Cura

26 de abril de 2026

Índice

Resumen	3
Abstract	3
Repositorios	3
1. Descripción del proyecto.....	4
1.1 Introducción.....	4
1.2 Contexto del proyecto.....	4
1.3 Objetivo del proyecto.....	4
1.4 Marco legal.....	5
2. Acuerdo del proyecto	6
2.1 Requisitos funcionales y no funcionales	6
2.2 Planificación de tareas y temporalización.....	7
2.3 Metodología a seguir	8
2.4 Temporalización	8
2.5 Presupuesto	9
2.6 Contrato y pliego de condiciones/licencia.....	9
2.7 Análisis de riesgos.....	10
3. Análisis y diseño	11
3.1 Modelado de datos	11
3.2 Análisis y diseño funcional.....	13
4. Implementación y pruebas	23
4.1 Implementación	23
4.2 Pruebas	30
5. Instalación, manual de usuario y documento de cierre	32
5.1 Instalación y documentación	32
5.2 Manual de usuario / ayuda integrada	35
6. Resultados y conclusiones.....	36

7. Cuaderno de bitácora	37
8. Bibliografía.....	38
Protección de datos y privacidad.....	38
Propiedad intelectual.....	38
Licencias de software.....	38
Tecnologías utilizadas	38
Sprite pack	38
Tutoriales en vídeo	38
Gestión de la base de datos.....	38
Elementos de la UI	38
9. Anexos	39
Diagrama de clases general: extensión.....	39
Diagrama de clases: Core del juego.....	40
Diagrama de clases: Items e Inventario	41
Diagrama de clases: Base de datos y persistencia	42
Diagrama de clases: Sistema de islas.....	43
Diagrama de clases: Menú principal.....	44
Diagrama de clases: UI e Inventario visual	45
Diagrama de clases: Configuración.....	46
Diagrama de flujo	47
Diagrama de estados (Ciclo de vida).....	48
Descomposición en módulos.....	49
Script de creación de la Base de datos	50

Resumen

ShinyVillage es un videojuego de rol en dos dimensiones desarrollado con Unity y C#. El jugador dirige a un personaje desde una perspectiva cenital, gestiona su inventario, interactúa con el mapa a través de un sistema de tiles y administra múltiples partidas guardadas en una base de datos local SQLite. El proyecto aplica una arquitectura en capas con patrones Singleton y Repository, orientada al aprendizaje del desarrollo de videojuegos en un contexto académico.

Abstract

ShinyVillage is a 2D role-playing game built with Unity and C#. The player controls a character from a top-down perspective, managing an inventory, interacting with a tile-based map, and handling multiple save files stored in a local SQLite database. The project follows a layered architecture using Singleton and Repository patterns, developed in an academic context as a practical exercise in video game programming.

Repositorios

[App principal \(ShinyVillage\)](#)

[App de instrucciones de instalación \(ShinyVillageTuto\)](#)

1. Descripción del proyecto

1.1 Introducción

ShinyVillage es un videojuego de rol en dos dimensiones (RPG 2D) creado con el motor Unity y programado en C#. La propuesta principal es poner al jugador en la dirección de una aldea, donde tiene que administrar los recursos, cultivar su granja y examinar el ambiente desde un punto de vista cenital típico del género.

El jugador maneja a un personaje que se mueve sin restricciones por el mapa, recoge objetos del entorno, los guarda en un inventario y realiza acciones con aquellos elementos del terreno que son interactivos, como las celdas agrícolas. El personaje es seguido de manera constante por la cámara. Como el progreso se almacena en una base de datos local (SQLite), el jugador tiene la posibilidad de crear múltiples partidas independientes, reanudarlas cuando quiera o eliminarlas desde el menú.

El ciclo de juego proyectado incorpora procedimientos relacionados con la agricultura (cultivar, regar y recoger productos en tiles), manejo del inventario (recoger, amontonar y soltar artículos) e interacción con el mapa a través de un sistema de tiles clasificados según su condición.

1.2 Contexto del proyecto

El proyecto se desarrolla en un contexto académico, sin un cliente externo, donde el propio equipo actúa como desarrollador y receptor técnico del producto. El entorno se basa en Unity con C#, utilizando SQLite integrado mediante NuGetForUnity para la persistencia de datos, mientras que el control de versiones se gestiona con Git mediante ramas de desarrollo.

La solución elegida estructura el juego en sistemas independientes conectados mediante patrones de diseño. Se utiliza el patrón Singleton en gestores como GameManager, DatabaseManager y SaveGameManager. La persistencia se gestiona con SQLite y el patrón Repository separa el acceso a la base de datos de la lógica del juego.

El juego contempla como único tipo de usuario el jugador. Su rol le permite crear y modificar partidas desde el menú, mover al personaje e interactuar con el mapa. Toda la gestión interna es transparente para el jugador y opera de forma automática en segundo plano.

1.3 Objetivo del proyecto

ShinyVillage tiene la finalidad de ofrecer al jugador una experiencia de ocio digital tranquilo, de estilo casual basado en una simple gestión de recursos y un entorno de fantasía. La aplicación no tiene vocación comercial en este momento porque está enmarcada en un contexto de aprendizaje y desarrollo en técnicas de programación de videojuegos.

1.4 Marco legal

Al tratarse de un videojuego de entretenimiento en fase de desarrollo académico, sin distribución comercial ni recogida de datos personales identificativos, el marco legal aplicable es reducido.

Protección de datos (RGPD / LOPDGDD)

La aplicación almacena únicamente datos de juego — nombre de personaje, progreso o posición — de forma local en el dispositivo del usuario, sin transmisión a servidores externos. En su estado actual no se tratan datos personales en el sentido del **RGPD (Reglamento UE 2016/679)** ni de la **LOPDGDD (LO 3/2018)**. Si en el futuro se incorporasen cuentas de usuario o cualquier dato identificativo, la aplicación quedaría sujeta a dichas normas.

Propiedad intelectual

El proyecto ShinyVillage se distribuye bajo una **licencia personalizada basada en Apache License 2.0**, a la que se añade una cláusula de restricción de uso comercial. Cualquier persona puede usar, estudiar, modificar y redistribuir el software, pero **no puede hacerlo con fines comerciales** sin autorización expresa. Esta licencia no está aprobada por la OSI al incluir dicha restricción, pero es plenamente válida desde el punto de vista legal.

Clasificación por edades

En caso de distribución pública, el sistema PEGI clasificaría previsiblemente el juego como PEGI 3, dado su contenido de fantasía sin elementos violentos ni adultos.

2. Acuerdo del proyecto

2.1 Requisitos funcionales y no funcionales

Requisitos funcionales

Capacidades que el programa debe poder ofrecer al usuario final, en este caso el jugador.

ID	Requisito
RF-01	El jugador puede crear una nueva partida introduciendo un nombre de personaje y un nombre de slot
RF-02	El jugador puede cargar una partida guardada previamente desde el menú principal
RF-03	El jugador puede borrar una partida existente, con confirmación previa
RF-04	El jugador puede sobrescribir una partida con el estado actual de la sesión en curso
RF-05	El personaje se desplaza por el mapa en las cuatro direcciones con animación asociada
RF-06	El jugador puede recoger objetos del mundo e incorporarlos al inventario
RF-07	El jugador puede consultar su inventario y eliminar objetos, que reaparecen en el mapa
RF-08	El jugador puede interactuar con tiles del mapa pulsando la tecla de acción
RF-09	El estado de la granja (tiles, cultivos, fases de crecimiento) se guarda y recupera por partida

Requisitos no funcionales

Este tipo de requisitos definen las condiciones y estándares de rendimiento y las restricciones del sistema.

ID	Requisito
RNF-01	Los datos de partida se almacenan localmente en el dispositivo del usuario mediante SQLite
RNF-02	El sistema de guardado persiste entre escenas sin pérdida de datos
RNF-03	La arquitectura debe permitir añadir nuevos sistemas (cultivos, NPCs) sin modificar el núcleo
RNF-04	El tiempo de carga de una partida guardada no debe ser perceptible para el usuario (Duración de menos de 5s)
RNF-05	El juego debe ejecutarse en PC con un rendimiento estable bajo Unity, sin crasheos ni problemas de rendimiento como quedarse “congelado”

2.2 Planificación de tareas y temporalización

El proyecto se organiza en varias fases bien diferenciadas que permiten avanzar de forma progresiva en su desarrollo.

En la Fase 1, centrada en los fundamentos del proyecto, se lleva a cabo la configuración inicial del entorno de Unity, así como la organización de la estructura de carpetas. Además, se implementa el sistema de movimiento del personaje junto con el seguimiento de cámara, y se desarrolla el uso de Tilemaps junto con las primeras interacciones básicas dentro del entorno del juego.

En la Fase 2, dedicada al sistema de inventario, se diseña la clase Inventory y toda la lógica relacionada con la gestión de slots, incluyendo la posibilidad de añadir, apilar y eliminar objetos. También se desarrolla la interfaz de usuario del inventario mediante componentes como Inventory_UI y Slot_UI, y se implementa la mecánica que permite soltar objetos al mundo.

La Fase 3 se enfoca en la persistencia de datos y la base de datos. En esta etapa se integra SQLite utilizando NuGetForUnity, se crea un DatabaseManager encargado de la conexión, la creación de tablas y las operaciones CRUD, y se desarrollan repositorios específicos como SaveSlotRepository

y PlayerRepository. Finalmente, se implementa el SaveGameManager como punto de entrada único para gestionar el guardado de la partida.

En la Fase 4, actualmente en desarrollo, se trabaja en el menú principal y la gestión de partidas. Esto incluye la creación del MainMenuManager, que controla acciones como iniciar una nueva partida, cargar partidas existentes o eliminarlas. También se desarrolla el SaveSlotUI, un prefab que representa cada partida con botones dinámicos, y se integra todo el flujo completo desde el menú hasta el juego y el sistema de guardado.

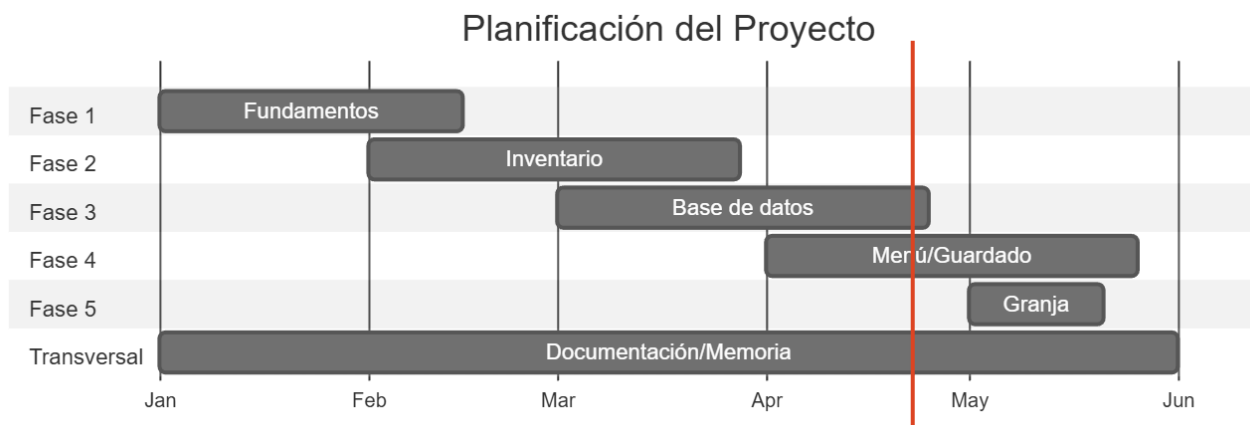
Por último, la Fase 5, también en desarrollo, aborda la implementación del sistema de granja. En ella se crean repositorios como FarmRepository y FarmTileRepository, se desarrolla la lógica de cultivo (plantar, regar y cosechar), y se garantiza la persistencia del estado de cada tile para cada partida.

2.3 Metodología a seguir

El proyecto adopta una metodología iterativa e incremental inspirada en Scrum, adaptada a un equipo de desarrollo pequeño y a un contexto académico. Cada fase equivale a un sprint con un entregable funcional al final. El desarrollo se gestiona mediante Git con ramas independientes por funcionalidad.

2.4 Temporalización

El siguiente diagrama refleja la distribución temporal estimada del proyecto:



Cada fase tiene una duración aproximada de dos a tres semanas, con solapamiento en la etapa de documentación, que se mantiene activa durante todo el desarrollo.

2.5 Presupuesto

En un contexto real de desarrollo profesional, este proyecto implicaría una serie de costes asociados tanto a los recursos humanos como a la infraestructura necesaria para su desarrollo.

El proyecto ha requerido una dedicación aproximada de 12 horas semanales desde el mes de enero hasta finales de curso (alrededor de 5 meses), lo que supone unas 240 horas de trabajo. Si se toma como referencia una tarifa media de un programador junior de aproximadamente 20 € por hora, el coste del desarrollo ascendería a una cantidad considerable.

Además, sería necesario contar con un equipo informático adecuado para trabajar con Unity de forma fluida, así como contemplar otros posibles costes relacionados con herramientas o recursos, aunque muchos de ellos pueden ser gratuitos.

A continuación, se muestra una estimación aproximada de los costes en un entorno real:

Concepto	Coste
Desarrollo de software (240h)	4.800 €
Equipo informático	1.000 €
SQLite + NuGetForUnity	0 €
Assets gráficos (libres de uso, CupNooble)	0 €
Control de versiones (Git y GitHub)	0 €
Unity (licencia personal)	0 €
Total	5.800 €

No obstante, en este caso concreto no ha existido ningún coste económico real, ya que el proyecto se ha desarrollado en un contexto académico, utilizando herramientas gratuitas o de uso libre, así como recursos previamente disponibles.

2.6 Contrato y pliego de condiciones/licencia

El proyecto se distribuye con una licencia Apache 2.0 con cláusula no comercial. Como se trata de un proyecto sin cliente externo, no hay contrato de prestación de servicios. El acuerdo de desarrollo queda recogido en los términos de entregas del centro formativo.

Riesgo	Probabilidad	Impacto	Mitigación
Corrupción de la base de datos SQLite	Baja	Alto	Cierre controlado de conexión en OnApplicationQuit y OnDestroy
Incompatibilidad de SQLite con la versión de Unity	Media	Alto	Uso de NuGetForUnity con versión fijada y probada
Deuda técnica por crecimiento no planificado	Media	Medio	Arquitectura en capas con patrón Repository desde el inicio
Pérdida de progreso en el repositorio Git	Baja	Alto	Ramas por funcionalidad y commits frecuentes
Falta de tiempo para completar el sistema de granja	Media	Medio	El núcleo es funcional sin él; se plantea como fase ampliable

2.7 Análisis de riesgos

A lo largo del desarrollo del proyecto, se prevé la aparición de posibles situaciones que puedan afectar tanto a la planificación como a la correcta implementación del mismo. Por ello, se ha realizado un análisis de riesgos con el objetivo de identificar aquellos eventos potenciales, evaluar su probabilidad e impacto, y definir medidas de mitigación que permitan reducir sus efectos. A continuación, se presenta una tabla con los principales riesgos detectados durante el desarrollo del proyecto.

3. Análisis y diseño

3.1 Modelado de datos

3.1.1 Modelo E/R

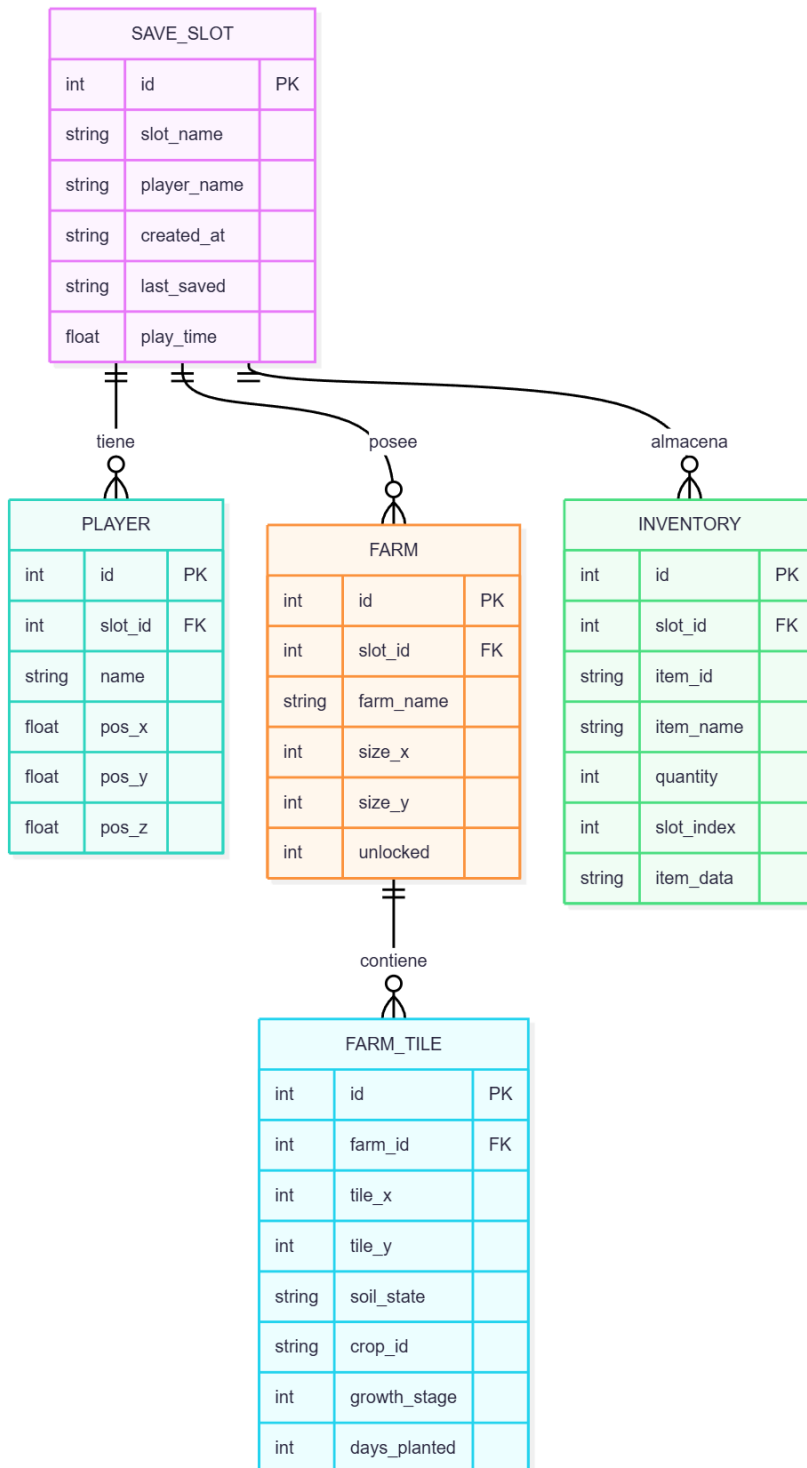


Figura 1: Diagrama E/R

3.1.2 Modelo relacional del sistema de guardado

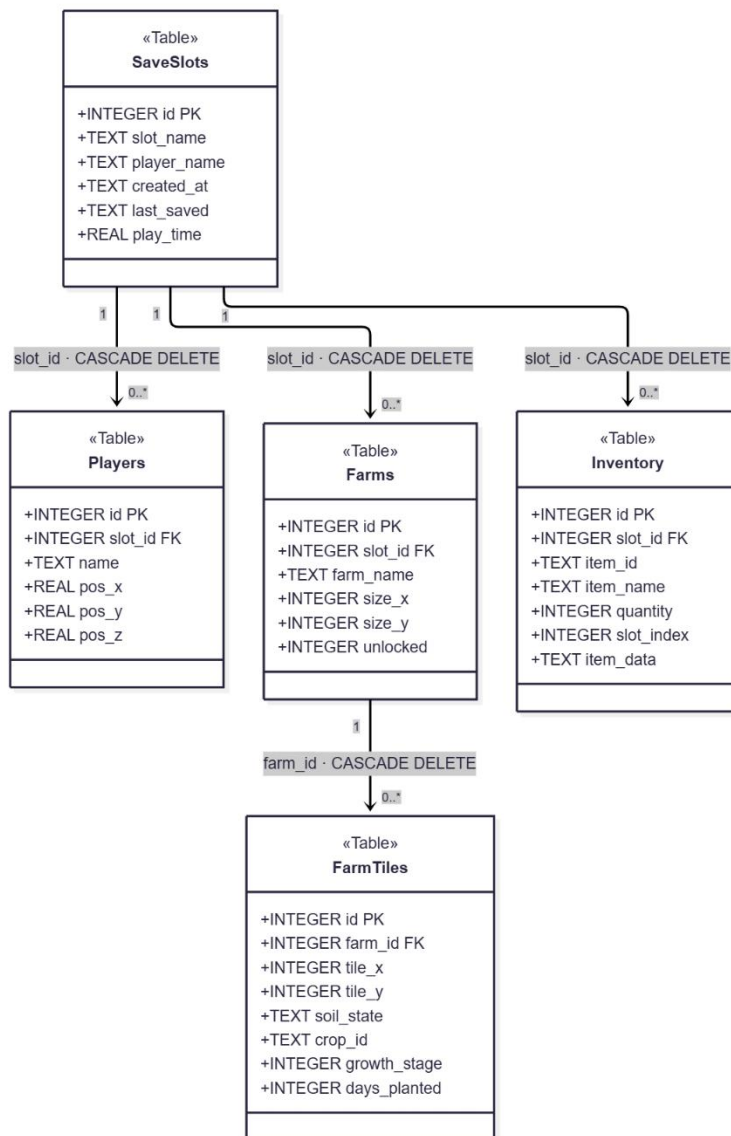


Figura 2: Modelo relacional

3.1.3 Script de la creación de la BBDD (SQLite): [Ver Anexos](#)

3.2 Análisis y diseño funcional

3.2.1 UML (Clases, secuencia y casos de uso)

Diagramas de clases

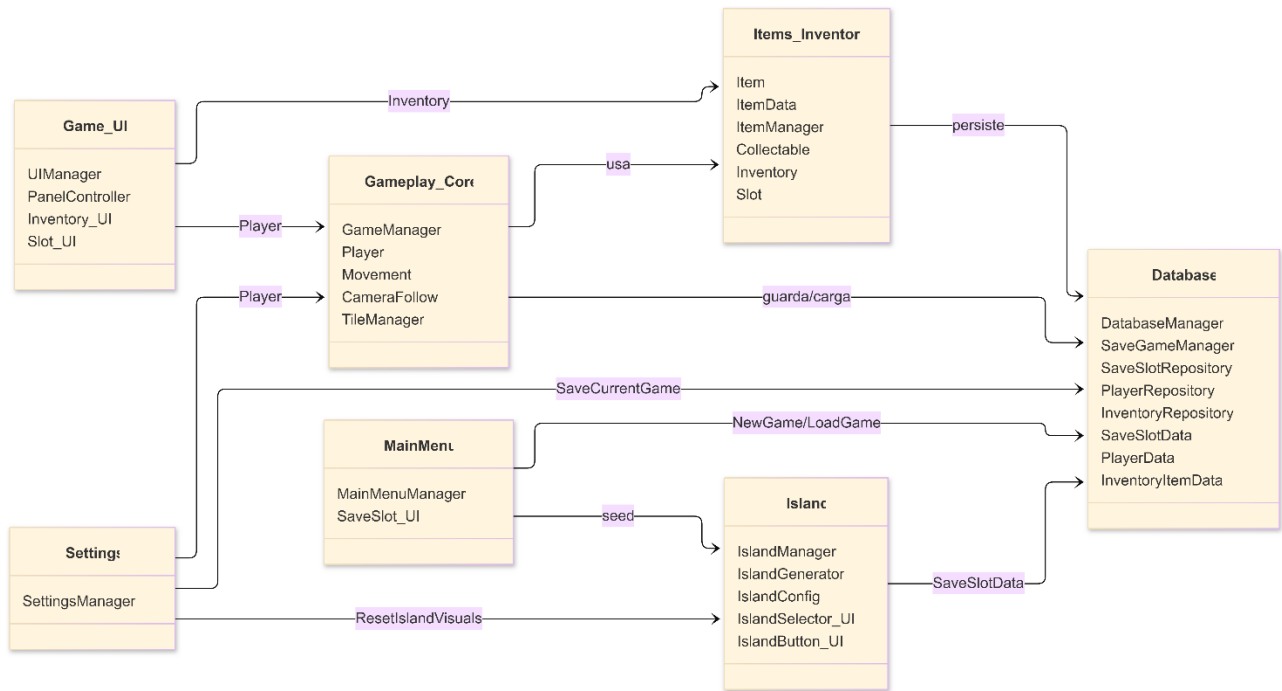


Figura 3: Diagrama de clases general

Diagrama de secuencia: Nueva partida

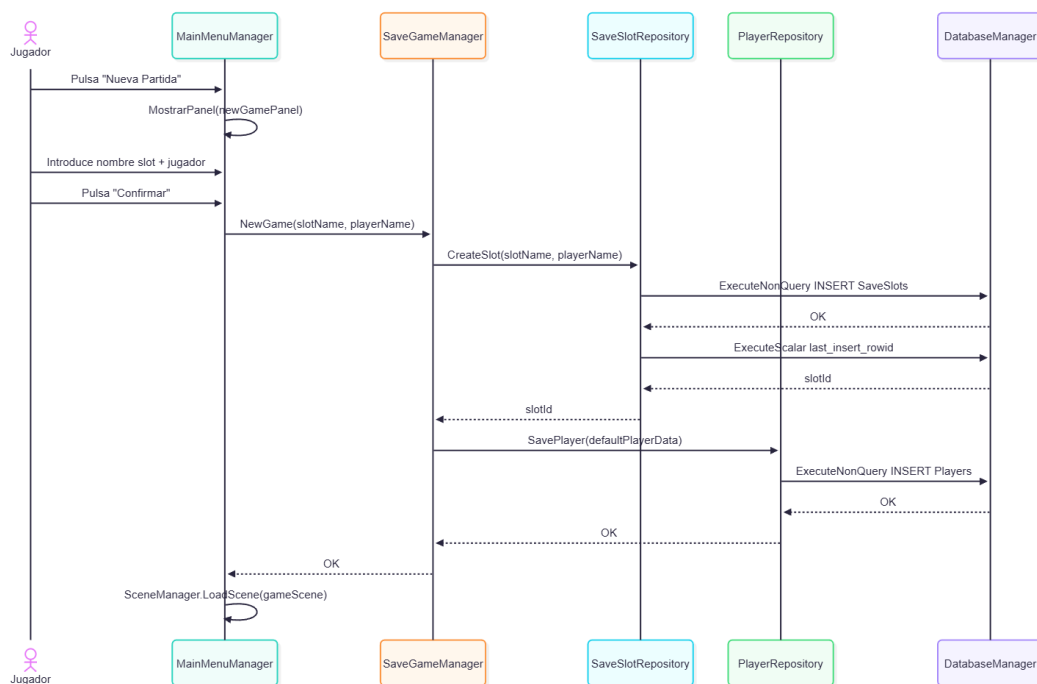


Figura 4: Diagrama de Secuencia para crear una nueva partida

El diagrama de secuencia ilustra el flujo completo de creación de una nueva partida en el sistema, involucrando la interacción entre seis componentes principales: el Jugador, MainMenuManager, SaveGameManager, SaveSlotRepository, PlayerRepository y DatabaseManager.

El proceso se inicia cuando el jugador pulsa el botón "Nueva Partida" en la interfaz principal. Esta acción desencadena que el **MainMenuManager** muestre el panel de nueva partida (NewGamePanel) al usuario. En este panel, el jugador introduce el nombre del slot de guardado y su nombre de jugador, tras lo cual pulsa el botón "Confirmar".

Una vez confirmada la entrada, el **MainMenuManager** invoca el método `NewGame(slotName, playerName)` en el **SaveGameManager**, pasándole ambos parámetros. El **SaveGameManager** actúa como orquestador del proceso de guardado y delega la creación del slot al **SaveSlotRepository** mediante la llamada `CreateSlot(slotName, playerName)`.

El **SaveSlotRepository** procede entonces a interactuar con la base de datos a través del **DatabaseManager**, ejecutando una consulta SQL de tipo INSERT mediante `ExecuteNonQuery INSERT SaveSlots`. Esta operación persiste el nuevo slot de guardado en la tabla correspondiente. El **DatabaseManager** confirma la operación exitosa devolviendo "OK" al **SaveSlotRepository**, el cual obtiene el identificador del slot recién creado (slotId) y lo retorna al **SaveGameManager**.

Con el slotId disponible, el **SaveGameManager** procede a crear los datos predeterminados del jugador invocando `SavePlayer(defaultPlayerData)` en el **PlayerRepository**. Este componente se encarga de persistir la información inicial del jugador ejecutando `ExecuteNonQuery INSERT Players` en el **DatabaseManager**. Una vez más, la base de datos confirma la operación con "OK", señal que se propaga de vuelta al **PlayerRepository** y posteriormente al **SaveGameManager**.

Finalmente, con ambas operaciones de base de datos completadas exitosamente, el **SaveGameManager** confirma al **MainMenuManager** que el proceso ha concluido correctamente mediante "OK". El **MainMenuManager** responde cargando la escena de juego apropiada con `LoadScene(gameScene)`, iniciando así la experiencia de juego para el usuario.

El diseño implementa un patrón de repositorio que separa la lógica de negocio de la capa de acceso a datos. Los componentes Repository (**SaveSlotRepository** y **PlayerRepository**) actúan como intermediarios especializados entre los managers de alto nivel y el **DatabaseManager**, proporcionando una abstracción clara sobre las operaciones de persistencia. Esta arquitectura facilita el mantenimiento, la testabilidad y permite modificar la implementación de almacenamiento sin afectar la lógica de negocio principal.

Diagrama de casos de uso

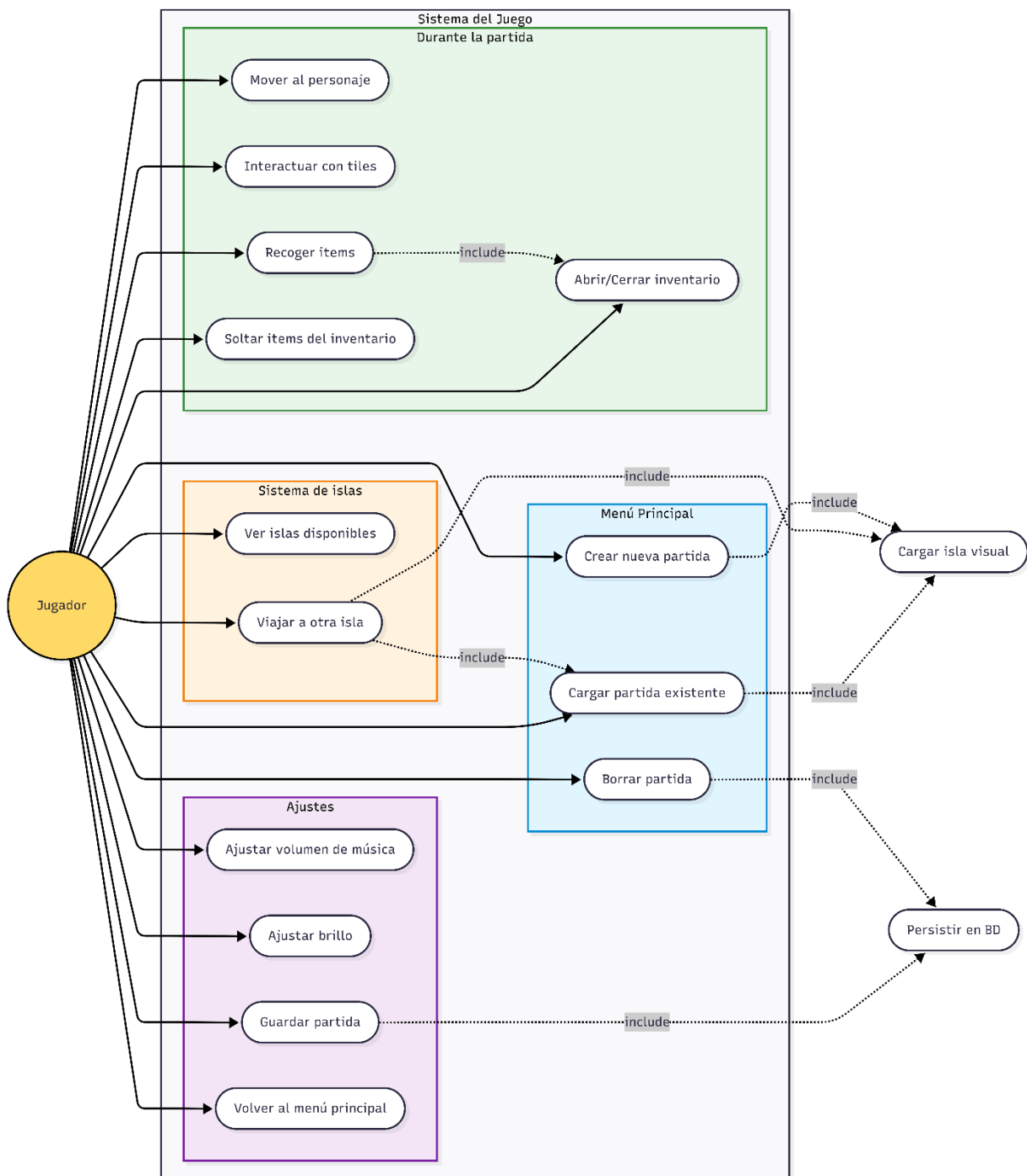


Figura 5: Diagrama de casos de Uso

El diagrama representa las acciones que un jugador puede realizar en el sistema, organizadas en cuatro bloques funcionales que coinciden con la arquitectura real del proyecto.

Actor principal. Existe un único actor, el *Jugador*, que interactúa con todos los casos de uso. No hay otros actores externos (no hay administrador, servidor remoto ni sistemas de terceros) porque el juego funciona de forma local contra una base de datos SQLite propia.

Menú Principal. Agrupa las acciones disponibles antes de entrar en partida: crear una nueva, cargar una existente o borrarla. Son la puerta de entrada al resto del sistema, ya que sin un slot activo el gameplay no se puede iniciar.

Gameplay. Recoge lo que el jugador hace durante la partida: moverse, interactuar con tiles del mapa, recoger ítems, abrir o cerrar el inventario y soltar objetos. Es la capa de interacción directa con el mundo del juego.

Sistema de islas. Representa la funcionalidad que permite ver todas las partidas guardadas como "islas" con su identidad visual propia y viajar entre ellas. Funciona como un puente entre el sistema de guardado y la visualización.

Ajustes. Reúne las acciones de configuración accesibles en cualquier momento desde el panel de settings: volumen de música, brillo, guardado manual y regreso al menú principal.

Relaciones include. Las flechas punteadas indican que un caso de uso depende de otro para completarse. Por ejemplo, *Cargar partida* y *Viajar a otra isla* incluyen siempre *Cargar isla visual*, porque no tiene sentido entrar en una partida sin aplicar su paleta de colores. De la misma forma, *Guardar partida* y *Borrar partida* incluyen *Persistir en BD*, que es la operación real contra SQLite. *Recoger ítems* incluye *Abrir/Cerrar inventario* porque tras la recogida se refresca la UI.

Lectura general. El diagrama muestra que todo el sistema gira en torno al concepto de *slot de partida*: casi todas las acciones importantes (cargar, viajar, guardar, borrar) terminan interactuando con la capa de persistencia, mientras que el gameplay y los ajustes son funcionalidades que dependen de tener un slot ya activo.

3.3 Análisis y diseño de la interfaz

3.3.1 Mockups

Para el diseño de los mockups se ha utilizado GoodNotes 6 en PadOS (IPad 10) debido a sus prestaciones como aplicación de notas y esbozos. En ella, se han realizado los mockups de las principales escenas y menús de la aplicación.

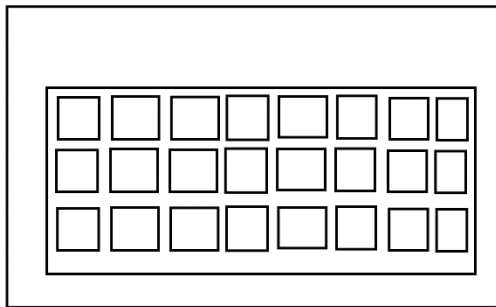


Figura 7: Mockup inventario



Figura 6: Mockup pantalla inicial

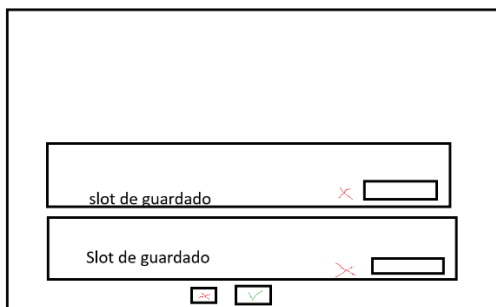


Figura 8: Mockup pantalla de carga de partidas

3.3.2 Diseño visual

El diseño visual utiliza sprites de estilo **Pixel Art** para una sensación retro y simplificada. El estilo con colores claros y en tonos pasteles le da un toque calmado y casual. El Asset Pack se obtiene desde [itch.io \(CupNooble\)](https://itch.io/CupNooble), que ofrece una versión gratuita muy completa.

Interfaz actual

Pantalla	Descripción
Inventario — img/Inventory.png	Panel de gestión de ítems con slots visuales
Menú principal — img/mainmenu.png	Pantalla de inicio con acceso a partidas

Cargar partida — img/load.png	Lista de slots guardados con acciones
Interfaz de juego — img/tilemap.png	Vista cenital del mapa con Tilemap

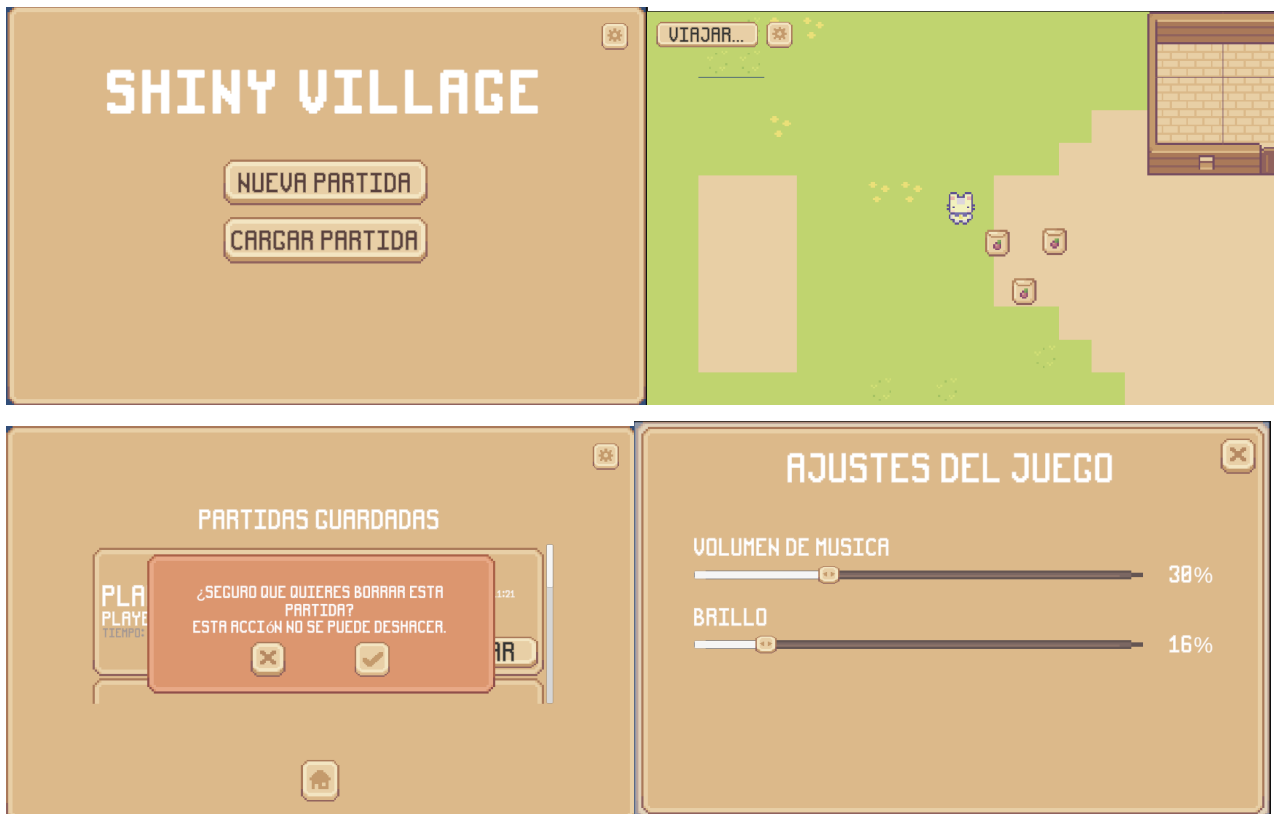


Figura 9: Interfaz del juego actual en estilo pixelart

3.4 Análisis y diseño de la arquitectura

3.4.1 Tecnologías usadas y herramientas

Herramienta	Rol
Unity	Motor de juego principal
C#	Lenguaje para los scripts
Visual Studio Code	IDE
Git / GitHub	Control de versiones y ramas de trabajo
Mermaid / Markdown	Documentación técnica del repositorio y diagramas
SQLite	Base de datos de persistencia
IPad + GoodNotes	Diagramas, mockups y notas

Motor de Juego — Unity

Unity se ha seleccionado como motor de juego por ser un entorno completo para desarrollar RPGs 2D. Posee sistemas integrados para el control de Tilemap, física 2D y UI Canvas. La alternativa inicial fue Godot 4, aunque el sistema de assets y la lógica 2D no están tan integradas y resultó complicado aprender a manejarlas al inicio.

Lenguaje: C# / .NET Standard

C# es el único lenguaje de scripting que soporta Unity. Se ha elegido frente al Visual Scripting porque puede ser versionado con Git de forma muy limpia y permite utilizar patrones como Singleton o Repository de manera explícita. C# es uno de los lenguajes estándar en la industria del videojuego junto a C++.

Entorno de desarrollo — VS Code

Se ha elegido VS Code frente a JetBrains Rider principalmente porque es gratuito y tiene integración fluida con Git y GitHub. Posee extensiones específicas para este proyecto: GitLens, MarkdownAllInOne y la extensión oficial de Unity.

La estrategia de ramas utilizada es: main (rama estable que recibe merges de funcionalidades completas) y feature-X (rama por funcionalidad en desarrollo).

Tecnologías para la documentación

Toda la documentación técnica se escribe en Markdown por su formato de texto plano, versionable con Git y con renderizado nativo en GitHub. La memoria del proyecto se expone en la [Wiki del repositorio](#) mediante un Workflow que tras cada commit actualiza la wiki de forma automatizada.

Paquetes y dependencias

NuGetForUnity

NuGetForUnity es un gestor de paquetes NuGet integrado en el Editor de Unity. Se usa para instalar System.Data.SQLite. La alternativa habitual (DLLs manuales en Assets/Plugins) es propensa a errores de versión y difícil de mantener.

SQLite

SQLite se emplea como motor de base de datos para el sistema de partidas guardadas. Al ser una solución embebida, no requiere servidor externo. Es mucho más robusta que PlayerPrefs, que solo admite tipos primitivos y no sobrevive a reinstalaciones. La arquitectura de acceso sigue el patrón Repository (SaveSlotRepository, PlayerRepository), que separa la lógica de negocio del acceso a datos.

New Input System

El nuevo Input System de Unity se emplea en lugar del sistema legacy (Input.GetKey) por ser el único que recibirá mantenimiento a largo plazo. Permite mapear controles de forma flexible desde un asset centralizado (InputActions)

3.4.2 Arquitectura de los componentes

Estructura lógica: capas y patrones

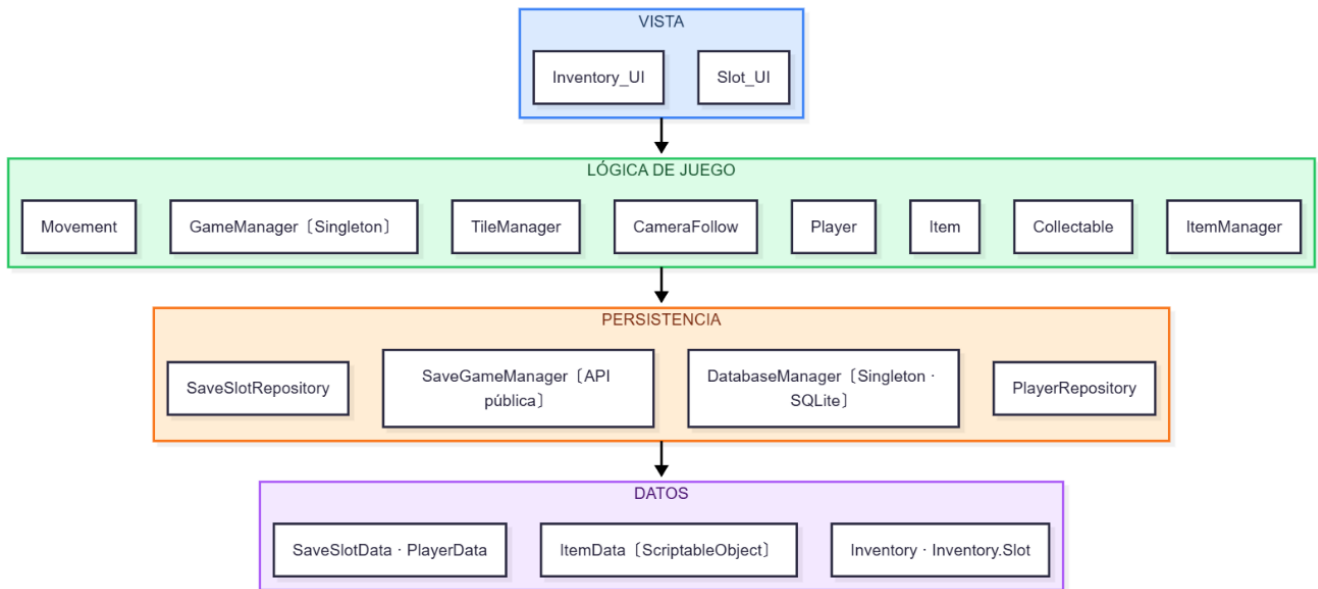


Figura 10: Diagrama de capas

El diagrama presenta una arquitectura de cuatro capas con flujo unidireccional descendente.

Capa de Vista

Contiene los componentes de interfaz de usuario: **Inventory_UI** y **Slot_UI**, responsables únicamente de la presentación visual al jugador.

Capa de Lógica de Juego

Implementa las mecánicas del juego con ocho componentes principales: **Movement** (desplazamiento), **GameManager** (controlador central Singleton), **TileManager** (sistema de casillas), **CameraFollow** (seguimiento de cámara), **Player** (entidad del jugador), **Item** y **Collectable** (objetos del mundo), e **ItemManager** (gestión de ítems).

Capa de Persistencia

Gestiona el almacenamiento de datos mediante cuatro componentes: **SaveSlotRepository** y **PlayerRepository** (operaciones CRUD), **SaveGameManager** (API pública de guardado) y **DatabaseManager** (Singleton SQLite que centraliza el acceso a base de datos).

Capa de Datos

Define las estructuras de información: **SaveSlotData + PlayerData** (información de partidas guardadas), **ItemData** (ScriptableObject con propiedades de ítems) e **Inventory.Slot** (estructura del inventario).

Cada capa depende exclusivamente de la inferior, garantizando separación de responsabilidades, bajo acoplamiento y facilidad de mantenimiento.

4. Implementación y pruebas

Este capítulo detalla la implementación técnica completa del proyecto ShinyVillage, organizado por subsistemas. Se presentan los fragmentos de código más relevantes con explicaciones detalladas de su funcionamiento, así como las pruebas realizadas para verificar el correcto comportamiento de cada componente.

4.1 Implementación

4.1.1 Sistema de Movimiento y Controles

El sistema de movimiento del jugador constituye la base de la interacción en el juego. Se implementó un sistema de movimiento en 8 direcciones con animaciones fluidas y física 2D.

Clase Movement- Movement.cs

El script Movement gestiona el desplazamiento del personaje y la sincronización con el sistema de animaciones. La implementación separa estratégicamente la captura de entrada del usuario del cálculo físico para garantizar un comportamiento consistente.

```
public class Movement : MonoBehaviour
{
    public float speed;
    public Animator animator;
    private Vector3 direction;

    private void Update()
    {
        // Se obtiene el input del jugador
        float horizontal = Input.GetAxisRaw("Horizontal");
        float vertical = Input.GetAxisRaw("Vertical");

        // Se crea y normaliza el vector de dirección para velocidad constante
        direction = new Vector3(horizontal, vertical).normalized;
        animateMovement(direction);
    }
}
```

La arquitectura del script distingue entre dos ciclos de actualización fundamentales en Unity. El método Update captura la entrada del jugador mediante GetAxisRaw, que proporciona valores discretos eliminando la suavización automática de Unity para lograr un control más responsivo. La normalización del vector dirección resulta crítica para corregir un problema común en juegos 2D: sin ella, el movimiento diagonal combinaría las magnitudes de ambos ejes, resultando en una velocidad aproximadamente 1.4 veces superior a la del movimiento cardinal.

4.1.2 Sistema de Inventario - Inventory.cs

El sistema de inventario implementa una lógica de apilamiento automático que optimiza el uso del espacio disponible. La clase Slot encapsula la gestión de cada espacio individual del inventario.

```
[System.Serializable]
public class Slot
{
    public string itemName;
    public int count;
    public int maxAllowed;
    public Sprite icon;

    public bool CanAddItem()
    {
        return count < maxAllowed;
    }

    public void AddItem(Item item)
    {
        this.itemName = item.data.itemName;
        this.icon = item.data.icon;
        count++;
    }
} ...
```

El atributo `System.Serializable` resulta fundamental para permitir que Unity serialice la clase en el Inspector y en el sistema de guardado, habilitando la persistencia de datos. El método `Add` implementa un algoritmo de dos pasadas que primero intenta apilar el ítem en slots existentes del mismo tipo con capacidad disponible, y solo si no encuentra espacio crea una nueva pila en un slot vacío. El método `RemoveItem` incluye lógica de limpieza que libera completamente el slot cuando la cantidad llega a cero, eliminando referencias al icono y al nombre para evitar estados inconsistentes en la interfaz.

4.1.3 Método DropItem - Player.cs

```
public void DropItem(Item item)
{
    Vector2 spawnLocation = transform.position;
    Vector2 spawnOffset = Random.insideUnitCircle * 2f;

    Item droppedItem = Instantiate(item, spawnLocation + spawnOffset,
    Quaternion.identity);
    droppedItem.rb2d.AddForce(spawnOffset * 0.2f, ForceMode2D.Impulse);
}
```

El método genera un efecto visual al soltar objetos mediante la combinación de posicionamiento aleatorio y física. `Random.insideUnitCircle` produce un vector bidimensional dentro de un círculo unitario, multiplicado por dos unidades para garantizar que el objeto aparezca fuera del collider del jugador y evite la recogida automática inmediata. La aplicación de fuerza con `ForceMode2D.Impulse` crea un impulso instantáneo que simula un lanzamiento suave, mejorando la retroalimentación visual de la acción.

4.1.4 Sistema de Tiles - *TileManager.cs*

```
public class TileManager : MonoBehaviour
{
    private Tilemap interactableMap;
    private Tile hiddenInteractableTile;
    private Tile InteractedTile;

    public bool IsInteractable(Vector3Int pos)
    {
        TileBase tile = interactableMap.GetTile(pos);
        return tile == hiddenInteractableTile;
    }

    public void SetInteracted(Vector3Int pos)
    {
        interactableMap.SetTile(pos, InteractedTile);
    }

    public Vector3Int GetCellPosition(Vector3 worldPos)
    {
        return interactableMap.WorldToCell(worldPos);
    }
}
```

El sistema utiliza dos tipos de tiles diferenciados para gestionar la interacción: `hiddenInteractableTile` marca posiciones interactivables de forma invisible, mientras que `InteractedTile` proporciona feedback visual al jugador tras la interacción. El método `WorldToCell` convierte coordenadas continuas del espacio mundial a coordenadas discretas del grid, permitiendo mapear con precisión la posición del jugador a celdas específicas del tilemap. La modificación del Tilemap en tiempo de ejecución mediante `SetTile` proporciona respuesta visual inmediata a las acciones del jugador.

4.1.5 Lógica de Interacción - Player.cs

```
void Update()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        Movement movement = GetComponent<Movement>();
        Vector3 lastDir = new Vector3(
            movement.animator.GetFloat("horizontal"),
            movement.animator.GetFloat("vertical")
        ).normalized;

        if (lastDir == Vector3.zero)
            lastDir = Vector3.down;

        Vector3Int cardinalDir;
        if (Mathf.Abs(lastDir.x) > Mathf.Abs(lastDir.y))
            cardinalDir = new Vector3Int((int)Mathf.Sign(lastDir.x), 0, 0);
        else
            cardinalDir = new Vector3Int(0, (int)Mathf.Sign(lastDir.y), 0);

        Vector3Int targetCell = playerCell + cardinalDir;

        if (GameManager.instance.tileManager.IsInteractable(targetCell))
            GameManager.instance.tileManager.SetInteracted(targetCell);
    }
}
```

El sistema recupera la última dirección directamente del Animator en lugar de recalcularla, garantizando coherencia con la orientación visual del personaje. La conversión a dirección cardinal elimina componentes diagonales, simplificando la interacción con el grid cuadrado del Tilemap mediante una comparación que determina si el movimiento es predominantemente horizontal o vertical. `Mathf.Sign` reduce cualquier valor a -1, 0 o 1, permitiendo la conversión directa a coordenadas de dirección cardinal.

4.1.6 Sistema de Base de Datos - DatabaseManager.cs

```
public List<Dictionary<string, object>> ExecuteQuery(string sql, SqlParameterizer
parameterize = null)
{
    var results = new List<Dictionary<string, object>>();

    using (IDbCommand cmd = _connection.CreateCommand())
    {
        cmd.CommandText = sql;
        parameterize?.Invoke(cmd);

        using (IDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                var row = new Dictionary<string, object>();
                for (int i = 0; i < reader.FieldCount; i++)
                    row[reader.GetName(i)] = reader.GetValue(i);
                results.Add(row);
            }
        }
    }

    return results;
}
```

El método `ExecuteQuery` implementa un patrón que extrae todos los resultados inmediatamente y los almacena en memoria, liberando la conexión de forma inmediata en lugar de mantenerla ocupada con un reader activo. El patrón `using` garantiza la liberación automática de recursos incluso en caso de excepción. Los resultados se almacenan en `Dictionary` indexados por nombre de columna para mayor legibilidad y menor propensión a errores que el acceso por índice numérico. El método `AddParameter` previene inyecciones SQL mediante parametrización, convirtiendo valores `null` de C# a `DBNull.Value` que representa correctamente la ausencia de valor en SQL.

4.1.7 Sistema de Islas - IslandGenerator.cs

```
public static IslandConfig GenerateIsland(int slotId, string islandName)
{
    Random.InitState(slotId * 1000);

    IslandConfig config = ScriptableObject.CreateInstance<IslandConfig>();
    config.slotId = slotId;
    config.islandName = islandName;

    config.groundColor = GenerateGroundColor();
}
```

```

    config.waterColor = GenerateWaterColor();
    config.accentColor = GenerateAccentColor();
    config.globalTint = GenerateGlobalTint();

    Random.InitState(System.DateTime.Now.Millisecond);
    return config;
}

private static Color GenerateGroundColor()
{
    float hue = Random.Range(0.08f, 0.35f);
    float saturation = Random.Range(0.3f, 0.6f);
    float value = Random.Range(0.5f, 0.8f);
    return Color.HSVToRGB(hue, saturation, value);
}

```

El sistema genera paletas de colores únicas para cada partida utilizando el ID del slot como semilla, garantizando que la misma partida siempre genere los mismos colores. La multiplicación por 1000 mejora la distribución de números aleatorios evitando que IDs consecutivos produzcan paletas similares. `ScriptableObject.CreateInstance` crea la configuración en memoria sin generar archivos en disco. La generación de colores utiliza el espacio HSV en lugar de RGB, permitiendo control preciso sobre la armonía cromática. El rango de Hue de 0.08 a 0.35 selecciona tonos naturales de tierra, evitando colores poco realistas para el terreno.

4.1.8 Sistema de Menú Principal - *MainMenuManager.cs*

```

public void OnCargarPartidaClick()
{
    MostrarPanel(loadGamePanel);
    StartCoroutine(RefrescarListaPartidasDelayed());
}

private IEnumerator RefrescarListaPartidasDelayed()
{
    yield return null;
    RefrescarListaPartidas();
}

```

La gestión de listas dinámicas en Unity presenta un desafío técnico: el motor requiere un frame completo para activar un panel y procesar su layout antes de instanciar elementos dentro. La corrutina con `yield return null` espera exactamente un frame, dando tiempo a Unity para calcular las dimensiones del panel. Sin esta espera, el `ContentSizeFitter` calcularía tamaños incorrectos al intentar medir contenido antes de la inicialización completa del panel.

El `VerticalLayoutGroup` no recalcula automáticamente la altura total al instanciar elementos dinámicamente. `LayoutElement.preferredHeight` informa al sistema cuánto espacio necesita cada

elemento. `Canvas.ForceUpdateCanvases` actualiza todos los Canvas de la jerarquía, mientras que `LayoutRebuilder.ForceRebuildLayoutImmediate` reconstruye el layout en el mismo frame en lugar de esperar al siguiente.

4.1.9 Componente *SaveSlot_UI* - *SaveSlot_UI.cs*

```
public void Setup(SaveSlotData data, MainMenuManager manager)
{
    _slotData = data;
    _menuManager = manager;

    slotNameText.text = data.SlotName;
    playerNameText.text = $"Player: {data.PlayerName}";
    lastSavedText.text = $"Last saved: {data.LastSaved:dd/MM/yyyy HH:mm}";

    loadButton.onClick.RemoveAllListeners();
    deleteButton.onClick.RemoveAllListeners();

    loadButton.onClick.AddListener(() =>
        _menuManager.OnLoadSlotClicked(_slotData.Id));
    deleteButton.onClick.AddListener(() =>
        _menuManager.OnDeleteSlotClicked(_slotData.Id, gameObject));
}
```

El método configura dinámicamente cada elemento de la lista de partidas. La string interpolation con formato de fecha aplica el formato directamente en la expresión. `RemoveAllListeners` resulta crucial para evitar acumulación de listeners duplicados al refrescar la lista, previniendo que las acciones se ejecuten múltiples veces. Las expresiones lambda capturan el valor actual del ID del slot, permitiendo que cada botón identifique correctamente qué partida debe cargar o eliminar.

4.2 Pruebas

4.1 Pruebas unitarias

Sistema de Movimiento

- **P1 - Velocidad diagonal:** Verifica que el personaje se mueve a la misma velocidad en diagonales que en direcciones cardinales. La normalización del vector evita que el movimiento diagonal sea un 41% más rápido.
- **P2 - Animaciones:** Comprueba que el sprite muestra la animación correcta según la dirección de movimiento (8 direcciones). El Blend Tree usa parámetros horizontal/vertical para seleccionar el sprite.
- **P3 - Cámara:** Asegura que la cámara sigue al jugador suavemente sin sacudidas. Ambos scripts usan FixedUpdate para mantener sincronización.

Sistema de Inventario

- **P4 - Apilamiento:** Confirma que objetos del mismo tipo se acumulan en un solo slot con contador en lugar de ocupar múltiples espacios.
- **P5 - Límite apilamiento:** Valida que al alcanzar el máximo permitido (99 unidades) el sistema crea automáticamente un nuevo slot para los objetos adicionales.
- **P6 - Retardo al soltar:** Previene que el jugador recoja inmediatamente un objeto que acaba de soltar. Se añade un offset aleatorio para alejarlo del trigger del personaje.
- **P7 - Actualización UI:** Verifica que la interfaz gráfica refleja exactamente el contenido del inventario tras cualquier cambio (recoger, soltar, abrir/cerrar).

Sistema de Tiles

- **P8 - Detección tiles:** Debe interactuar solo con tiles marcados como interactuables, pero actualmente modifica tiles adyacentes no deseados.
- **P9 - Dirección interacción:** - Debe modificar únicamente el tile en la dirección que mira el jugador, pero la detección direccional falla.

Sistema de Base de Datos

- **P12 - Creación partida:** Confirma que se genera correctamente un nuevo registro con ID único, semilla de isla, y datos iniciales del jugador en las tablas correspondientes.
- **P13 - Carga partida:** Verifica que se recuperan todos los datos guardados (nombre, posición, inventario) y se restaura el estado exacto de la sesión anterior.
- **P14 - Eliminación CASCADE:** Valida que al borrar una partida se eliminan automáticamente todos sus datos relacionados sin dejar registros huérfanos en la base de datos.

Sistema de Islas

- **P15 - Consistencia colores:** Asegura que una misma partida genera siempre la misma paleta de colores independientemente de cuándo se cargue. Usa la semilla para inicializar el generador aleatorio.
- **P16 - Unicidad colores:** Confirma que cada partida nueva tiene colores visualmente diferenciables de las demás, facilitando la identificación visual en el menú de carga.

Interfaz Menú

- **P17 - Validación formulario:** Previene la creación de partidas con nombre vacío o datos incompletos, mostrando mensaje de error apropiado.
- **P18 - ScrollView dinámico:** Verifica que el contenedor de la lista de partidas se redimensiona automáticamente cuando hay más elementos que el espacio visible, permitiendo scroll vertical.
- **P19 - Prevención listeners:** Evita que los botones acumulen múltiples eventos al refrescar la lista, lo que causaría que una acción se ejecute varias veces.

4.2 Pruebas de integración

- **P10 - Flujo completo:** Simula una sesión real de juego ejecutando todas las funcionalidades en conjunto (movimiento, inventario, interacción, menús) para detectar conflictos entre sistemas. Valida también el rendimiento sostenido.
- **P11 - Persistencia estado:** Confirma que el inventario mantiene su contenido al cerrar y reabrir durante la misma sesión (base para el sistema de guardado futuro).

5. Instalación, manual de usuario y documento de cierre

5.1 Instalación y documentación

5.1.1 Requisitos del Sistema

Antes de detallar la instalación, se exponen las condiciones técnicas mínimas que debe cumplir el equipo del usuario para garantizar una ejecución estable. Estos requisitos derivan directamente de las decisiones tecnológicas del proyecto: el motor Unity, la programación en C# sobre .NET, la persistencia mediante SQLite y el carácter 2D del juego.

Sistema Operativo: Windows 10 (64-bit) / macOS 10.13+ / Ubuntu 18.04+ Se opta por soporte multiplataforma porque Unity permite compilar el mismo proyecto para los tres sistemas sin reescribir código. Las versiones indicadas son las mínimas soportadas por el motor y por la biblioteca nativa de SQLite, que además exige arquitectura de 64 bits.

Procesador: Intel Core i3 o equivalente Resulta suficiente al tratarse de un RPG 2D con vista cenital, sin cálculos 3D ni físicas complejas. Las tareas las asume sin problema cualquier CPU moderna de esta gama.

RAM: 4 GB Valor alineado con las recomendaciones de Unity para proyectos 2D ligeros. Cubre la carga de escenas, sprites, los `ScriptableObject` del inventario y la conexión abierta a SQLite, dejando margen para el sistema operativo.

Gráficos: Tarjeta compatible con DirectX 10 Es la API mínima que Unity utiliza para renderizar sus escenas en Windows, incluso en 2D. En macOS y Linux el motor recurre a Metal y Vulkan/OpenGL, por lo que cualquier GPU integrada de la última década cumple el requisito.

Almacenamiento: 500 MB disponibles Espacio destinado al ejecutable, los recursos gráficos, las bibliotecas nativas de SQLite de cada plataforma y el archivo `savegame.db` generado en `Application.persistentDataPath`, que crece según avanza la partida.

.NET Framework 4.7.2 o superior (Windows) Requisito derivado del paquete `System.Data.SQLite`, que depende de esta versión para gestionar el guardado y la carga de partidas. En macOS y Linux se resuelve de forma transparente mediante el entorno Mono integrado por Unity.

En conjunto, estos requisitos buscan que el juego sea accesible en equipos de gama media-baja modernos, manteniendo la robustez que aporta una arquitectura basada en Unity, C# y una base de datos relacional local.

5.1.2 Instalación para Usuarios Finales

Proceso de Instalación:

- Descargar el instalador `ShinyVillage_Setup.exe` (Windows) o `ShinyVillage_Setup.dmg` (macOS)
- Ejecutar el instalador con permisos de administrador

Primera Ejecución:

El juego crea automáticamente la base de datos SQLite en: `%AppData%\ShinyVillage\saves.db` y se generan archivos de configuración en: `%AppData%\ShinyVillage\settings.json`

5.1.3 Configuración para Desarrolladores

Proceso de Configuración:

Clonar el repositorio:

```
git clone https://github.com/little-shiny/ShinyVillage.git  
cd ShinyVillage
```

Abrir el proyecto en Unity:

Unity Hub → Add → Seleccionar carpeta del proyecto

Verificar que la versión de Unity coincida (2022.3 LTS)

Configurar el entorno:

El proyecto usa Package Manager para dependencias

Unity descargará automáticamente: TextMeshPro, Universal RP

Verificar en `Window → Package Manager` que todos los paquetes estén instalados

Estructura de carpetas:

Assets/

```
|— Scenes/           # Escenas principales (MainMenu, GameScene)
|— Scripts/          # Código C# organizado por sistemas
|— Prefabs/          # Prefabs de tiles, items, UI
|— Sprites/          # Sprites del personaje, tiles, objetos
|— Database/         # Scripts de gestión SQLite
└— StreamingAssets/ # Base de datos SQLite
```

5.1.4 Base de Datos

Inicialización Automática:

Al ejecutar `DatabaseManager.InitializeDatabase()` se crea `saves.db` con las tablas:

- `SaveSlots`: Partidas guardadas
- `Players`: Datos del jugador (posición, nombre)
- `Inventory`: Objetos del inventario
- `FarmTiles`: Estado de los tiles de cultivo

Ubicación de la Base de Datos:

- **Build final**: `[RutaEjecutable]/StreamingAssets/saves.db`
- **Editor Unity**: `Assets/StreamingAssets/saves.db`

5.2 Manual de usuario / ayuda integrada

Para facilitar el aprendizaje del juego, se ha desarrollado una aplicación multiplataforma que funciona como tutorial interactivo y manual de usuario de Shiny Village. Se puede descargar desde el repositorio en la [carpeta releases](#) (de momento solo disponible para Windows como release pero preparada para versión web y app)

5.2.1 Tecnología Empleada

La aplicación se ha implementado utilizando **Flutter**, un framework de Google que permite compilar para Android, iOS, Windows, macOS, Linux y Web desde una única base de código.

5.2.2 Estructura y Funcionalidades

La aplicación se divide en cuatro módulos principales:

- **Pantalla Principal:** Menú de navegación con la estética visual del juego
- **Tutorial Interactivo:** Páginas navegables con capturas de pantalla del juego y anotaciones interactivas que explican cada elemento de la interfaz
- **Guía de Controles:** Referencia organizada de comandos de teclado agrupados por categorías (movimiento, interacción, menús)
- **Manual Completo:** Secciones expandibles con información detallada sobre mecánicas, guardado y consejos estratégicos

5.2.3 Implementación Técnica

El código sigue la arquitectura recomendada por Flutter, utilizando widgets reutilizables y navegación mediante `Navigator`. Las capturas de pantalla se almacenan localmente en `assets/tutorial/` para funcionamiento offline, y la interfaz se adapta automáticamente a diferentes tamaños de pantalla.

La compilación multiplataforma se realiza mediante los comandos `flutter build` específicos para cada plataforma (`apk`, `ios`, `windows`, `web`), generando ejecutables nativos optimizados. El archivo `pubspec.yaml` configura dependencias, `assets` y parámetros de compilación, incluyendo la generación automática de iconos mediante `flutter_launcher_icons`.

6. Resultados y conclusiones

El proyecto ha alcanzado satisfactoriamente los objetivos técnicos principales establecidos. El sistema de movimiento funciona correctamente con velocidad normalizada en las ocho direcciones, animaciones sincronizadas y cámara fluida. El inventario implementa apilamiento automático, límites por ranura y sincronización perfecta con la interfaz gráfica. La persistencia mediante SQLite almacena correctamente las partidas con eliminación en cascada que mantiene la integridad referencial. El sistema de generación de islas únicas mediante semillas aleatorias funciona impecablemente, proporcionando consistencia visual entre sesiones y diferenciación clara entre partidas.

Sin embargo, el sistema de tiles interactivables presenta deficiencias significativas en la detección direccional y modifica tiles adyacentes no deseados, requiriendo refactorización completa. El menú de ajustes permanece incompleto, faltando implementar las funcionalidades de audio y resolución. Además, el juego no implementa el guardado automático y se pierde el progreso si el jugador no guarda la partida desde el menú.

El desarrollo ha proporcionado aprendizajes valiosos en integración de bases de datos con Unity, normalización de vectores para movimiento, sincronización de ciclos de actualización para evitar artefactos visuales, gestión de interfaces dinámicas y uso de semillas para reproducibilidad. La separación entre lógica y visualización, junto con el testing incremental, ha demostrado su utilidad para detectar errores tempranamente y facilitar el mantenimiento.

Las principales dificultades resueltas incluyeron problemas de redimensionamiento de scroll views, recogida inmediata de objetos soltados, superposición de paneles y acumulación de listeners en botones.

El proyecto presenta fortalezas notables como arquitectura modular escalable, sistema de persistencia robusto, rendimiento estable superior a treinta fotogramas por segundo, código documentado y el sistema de islas únicas funcionando perfectamente. Las áreas de mejora incluyen la refactorización del sistema de tiles, ampliación del contenido de gameplay, mejora de retroalimentación visual y auditiva, y refinamiento estético de la interfaz.

En conclusión, se ha logrado crear una base funcional para un juego de gestión agrícola. Los sistemas fundamentales de persistencia, inventario y movimiento están correctamente implementados. Aunque el sistema de tiles presenta errores, estos son técnicamente solucionables sin comprometer la arquitectura general. El proyecto cumple su función como demostración técnica y proporciona una plataforma excelente para continuar el desarrollo añadiendo mecánicas de gameplay más complejas. Los conocimientos adquiridos en normalización vectorial, sincronización

de sistemas y gestión de datos persistentes son directamente aplicables a proyectos futuros de mayor envergadura.

Como posibles mejoras y futuras implementaciones se podrían incluir sistemas de guardado de los datos en la nube, así como una base de datos donde se encontraran todos los tipos de cultivos y objetos del juego, facilitando la gestión de cara a añadir y escalarlo a mayores funcionalidades.

Otra implementación que no fue posible debido a la falta de tiempo fue una idea de asignación de “skins” a los diversos jugadores mediante una tabla de la base de datos, de manera que se seleccionaran de forma aleatoria y sin repetirse 3 sprites diferentes para cada jugador y que pudiesen intercambiarse mediante la interfaz del juego.

7. Cuaderno de bitácora

El proyecto se desarrolló durante diez semanas con seguimiento semanal de tareas y comparativa con la planificación inicial. Las primeras dos semanas se dedicaron al sistema de movimiento y cámara, completándose según lo previsto con normalización de vectores, animaciones mediante Blend Tree y seguimiento fluido de cámara.

Las semanas tercera y cuarta abordaron el inventario, experimentando la primera desviación al consumir veinticinco por ciento más tiempo del planificado por problemas de recogida inmediata y sincronización de interfaz. La quinta semana recuperó el retraso implementando exitosamente el sistema de persistencia SQLite con todas sus funcionalidades. La sexta semana desarrolló el menú principal pero requirió cincuenta por ciento más tiempo por problemas no anticipados de scroll view y listeners duplicados.

La séptima semana resultó especialmente productiva, completando el sistema de islas únicas en la mitad del tiempo estimado. La octava semana marcó la desviación crítica con el sistema de tiles interactivables que, aunque consumió el tiempo planificado, quedó con errores graves requiriendo refactorización completa. La novena semana sufrió las consecuencias, dejando incompleto el menú de ajustes al priorizar intentos de corrección del sistema de tiles. La décima semana se dedicó íntegramente a pruebas exhaustivas y documentación de resultados.

El proyecto se completó en las diez semanas estimadas pero con funcionalidades incompletas. Los objetivos técnicos fundamentales de movimiento, inventario, persistencia e islas únicas se cumplieron satisfactoriamente en plazo. Sin embargo, se subestimó la complejidad del sistema de tiles y los tiempos necesarios para interfaces dinámicas en Unity. El menú de ajustes y el sistema de tiles quedaron pendientes de finalización completa. La temporalización global fue acertada en duración total, aunque la distribución interna de tareas requirió ajustes durante el desarrollo.

8. Bibliografía

Protección de datos y privacidad

- [Reglamento \(UE\) 2016/679 — RGPD](#)
- [Ley Orgánica 3/2018 — LOPDGDD](#)
- [AEPD — Guía para el cumplimiento del RGPD](#)

Propiedad intelectual

- [Real Decreto Legislativo 1/1996 — Ley de Propiedad Intelectual](#)

Licencias de software

- [Apache License, Version 2.0 — texto oficial](#)
- Open Source Initiative — definición de licencia de código abierto

Tecnologías utilizadas

- [Unity — documentación oficial](#)
- [SQLite — documentación oficial](#)
- [System.Data.SQLite — documentación del paquete](#)
- [Patrón Singleton en Unity — Game Programming Patterns](#)

Sprite pack

- [Sprout Lands Asset Pack — CupNooble](#)

Tutoriales en vídeo

- Awesome Tuts (2021). Learn Unity - Beginner's Game Development Tutorial
- [Digestible \(2021\). Using SQLite in Unity](#)
- [JacquelynneHei \(2022\). How to Make a 2D RPG in Unity](#)
- Sunny Valley Studio (2022). Using Tools - Farm Game In Unity Devlog 2

Gestión de la base de datos

- [Sistem.Data.SQLite Documentation](#)
- [Using SQLite in Unity - Digestible](#)

Elementos de la UI

- `__Vertical Layout`

Diagrama de clases general: extensión

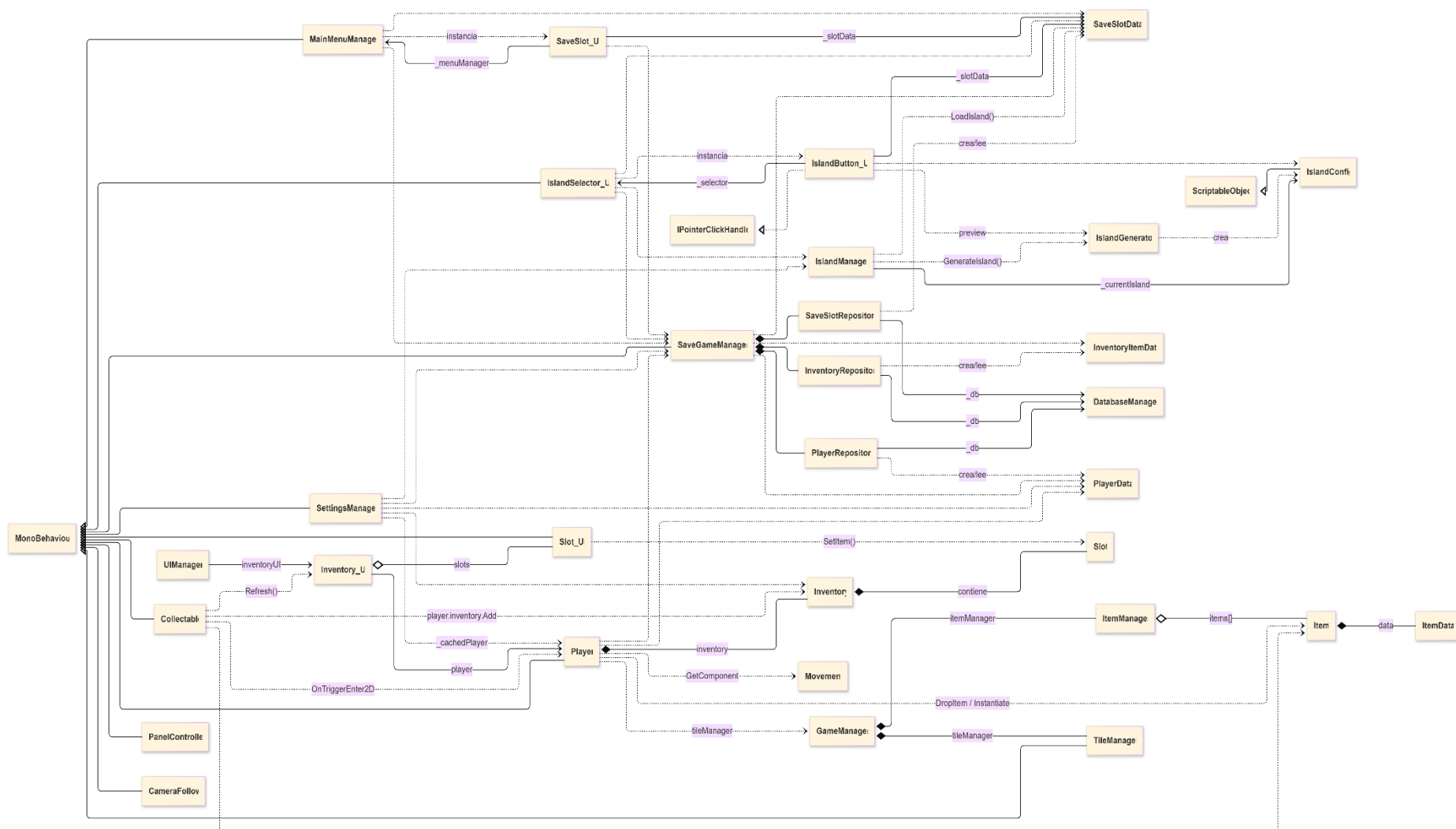


Diagrama de clases: Core del juego

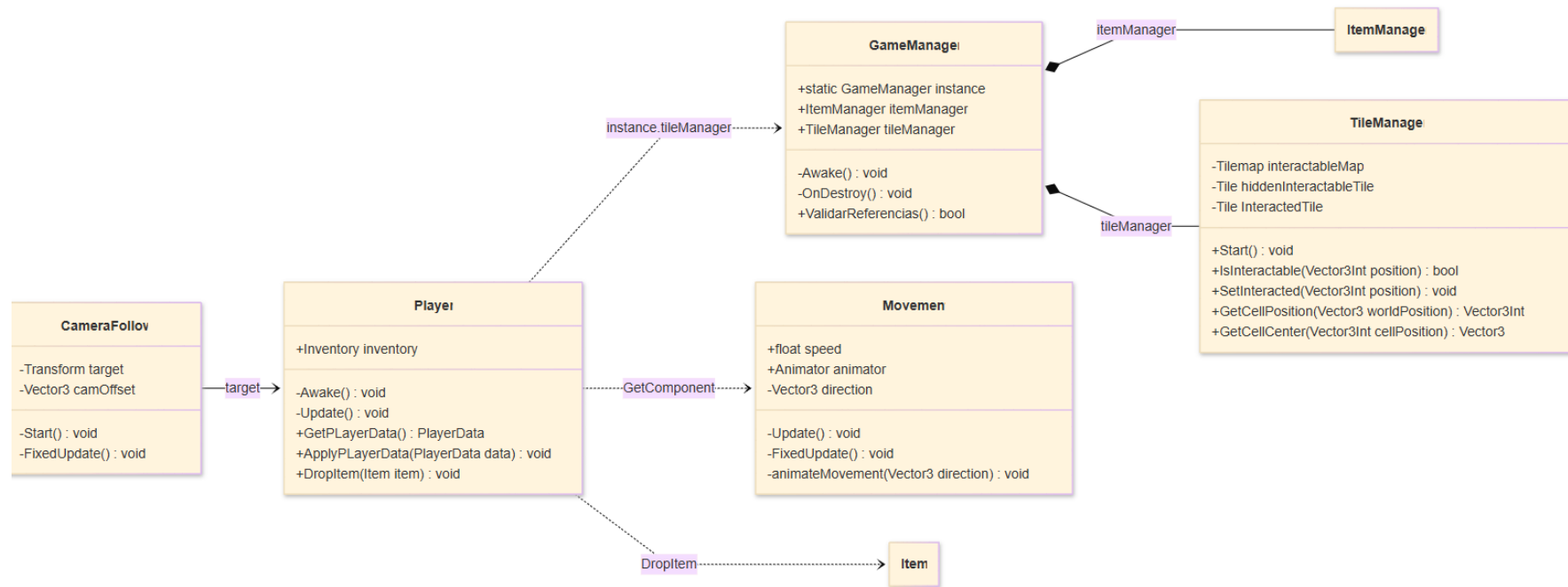


Diagrama de clases: Items e Inventario

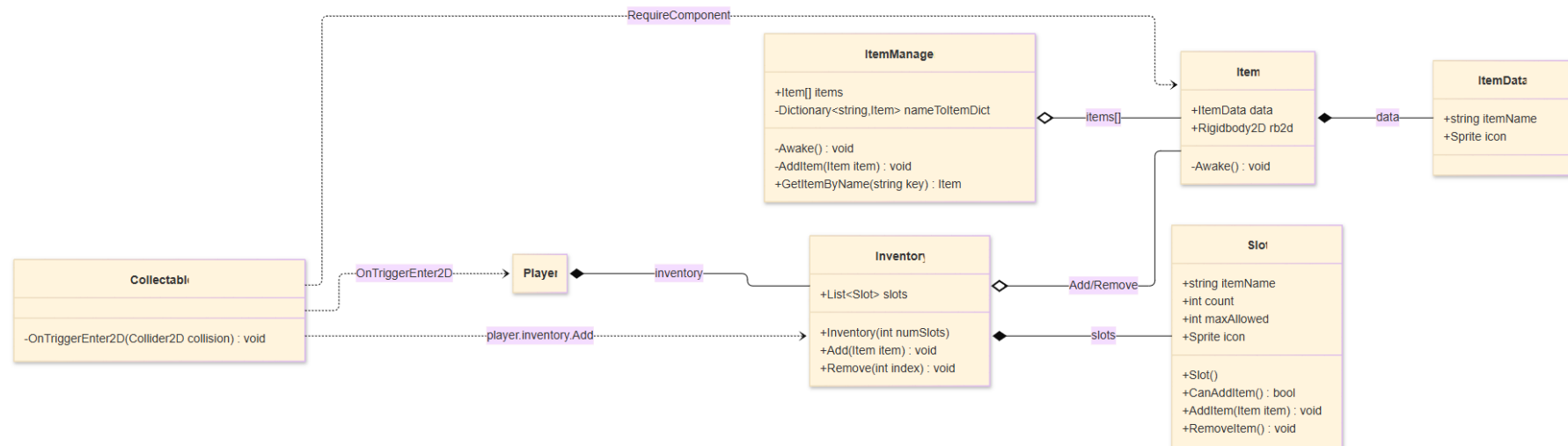


Diagrama de clases: Base de datos y persistencia

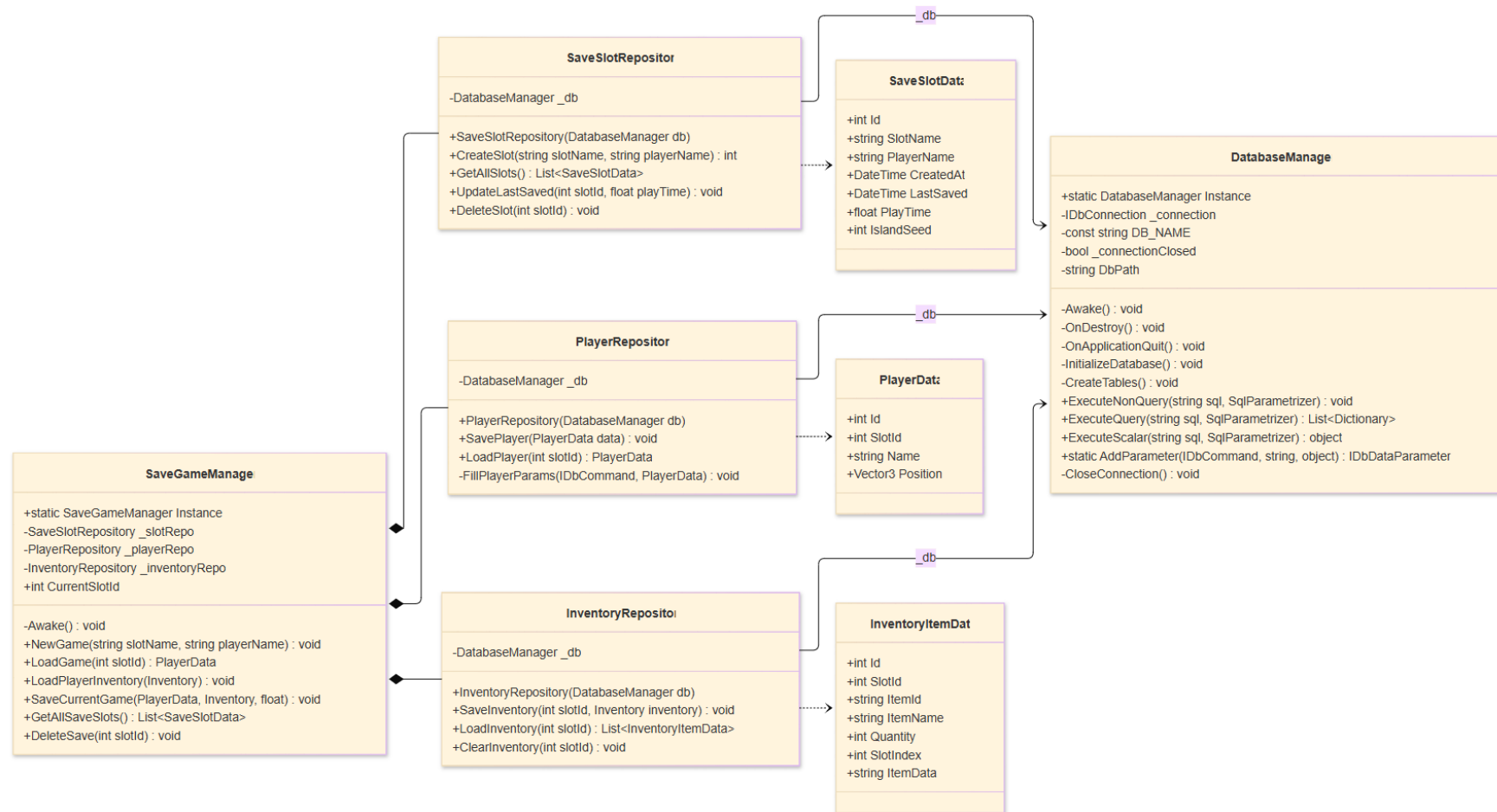


Diagrama de clases: Sistema de islas

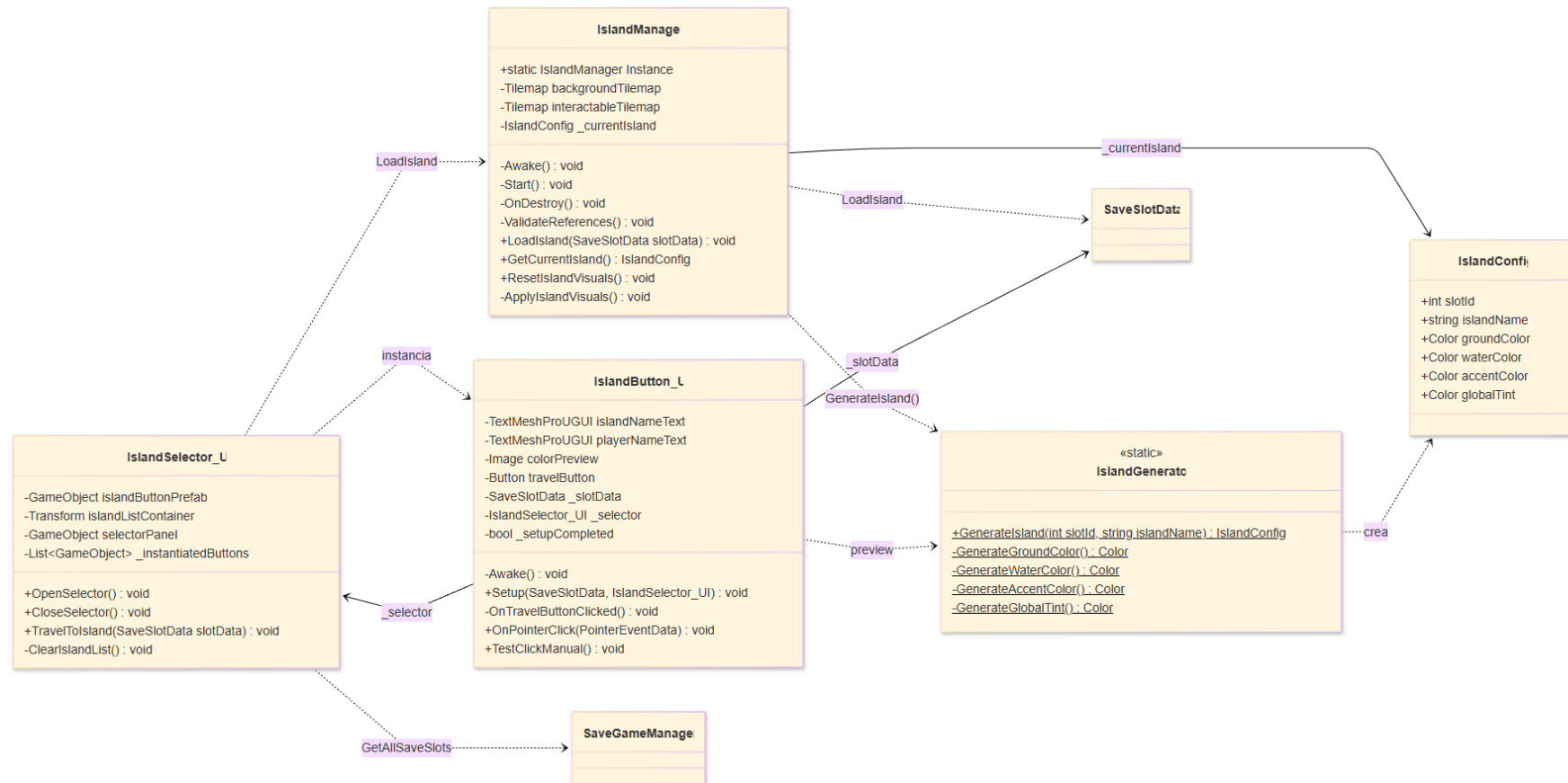


Diagrama de clases: Menú principal

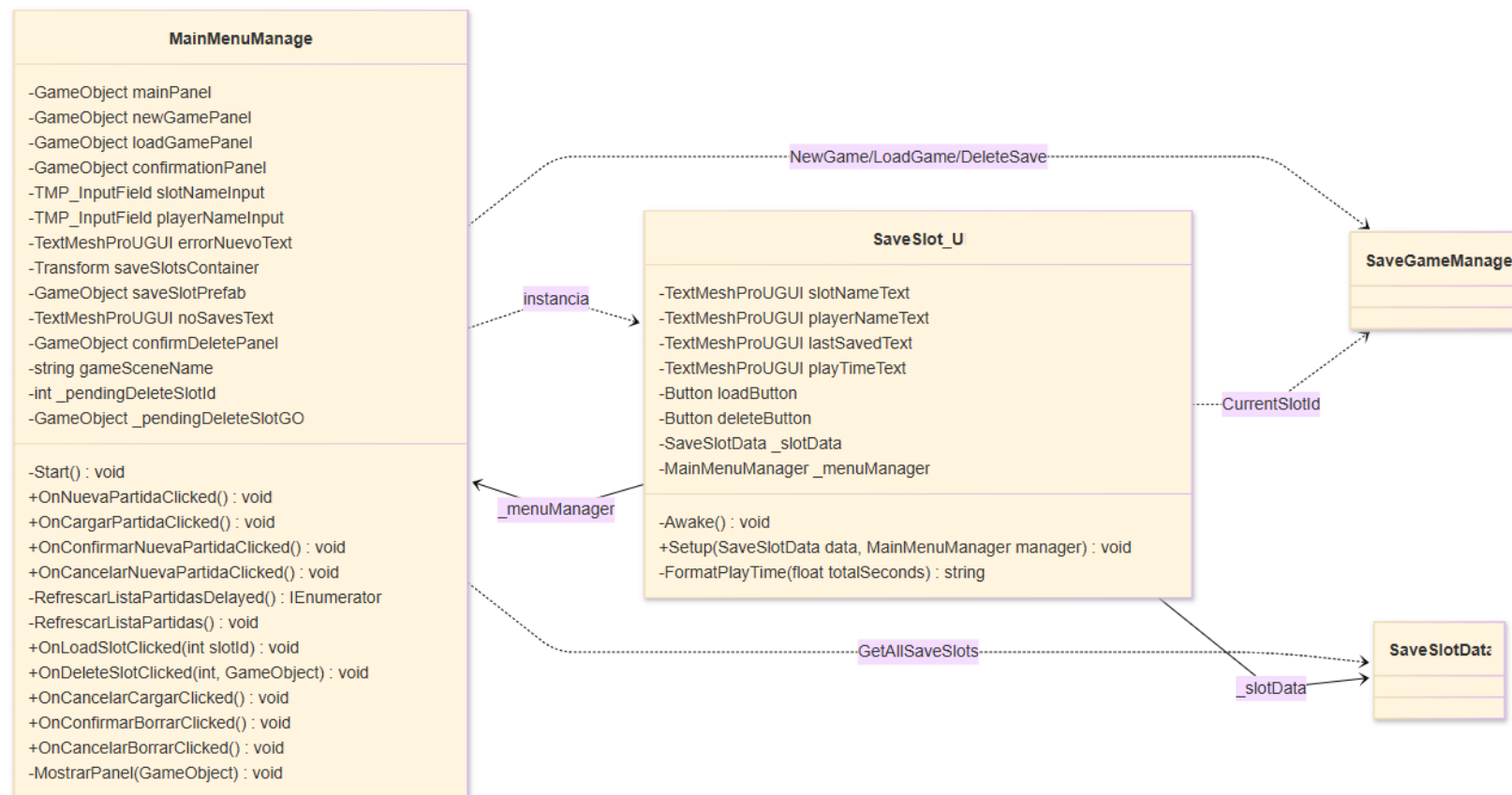


Diagrama de clases: UI e Inventario visual

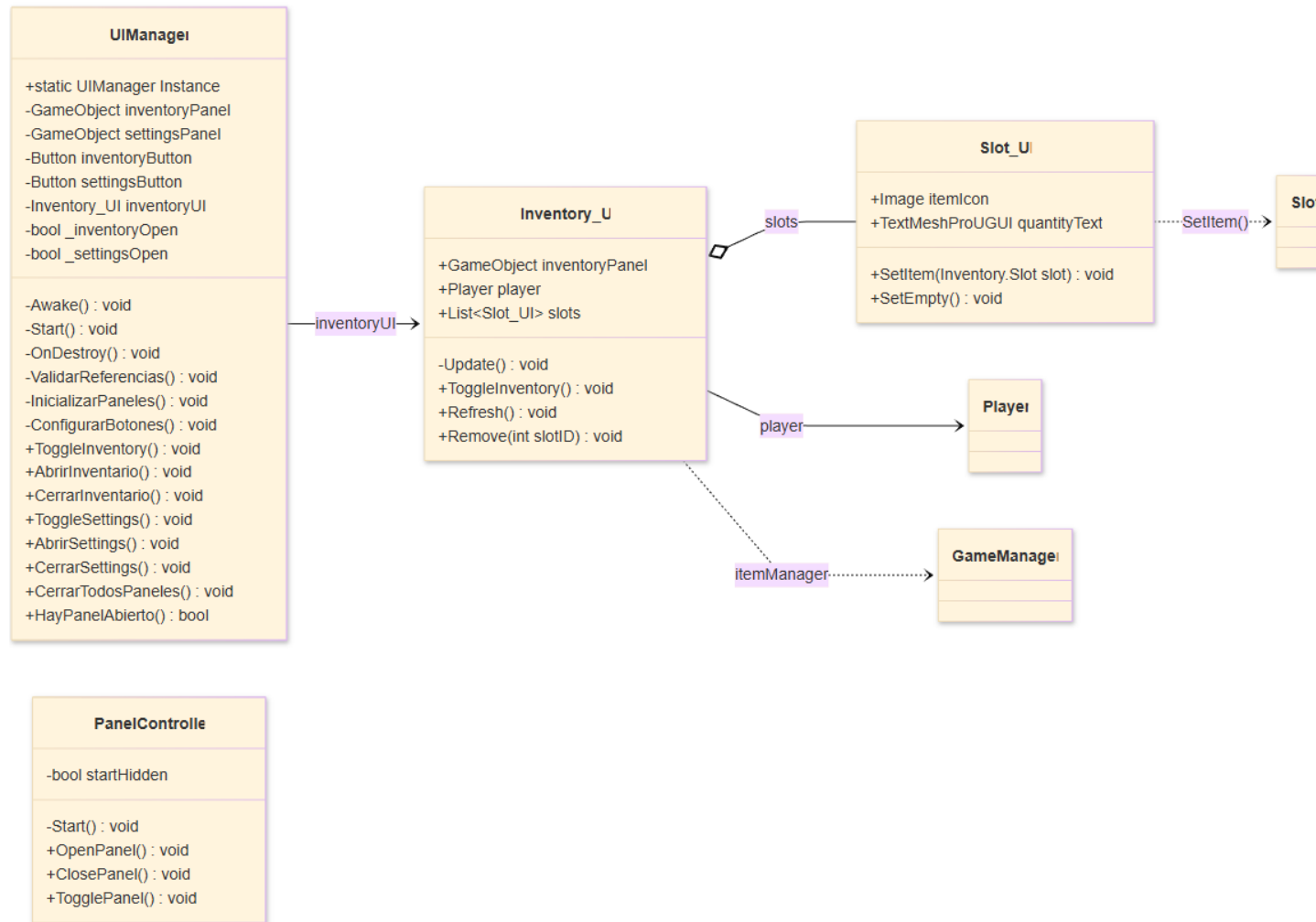


Diagrama de clases: Configuración

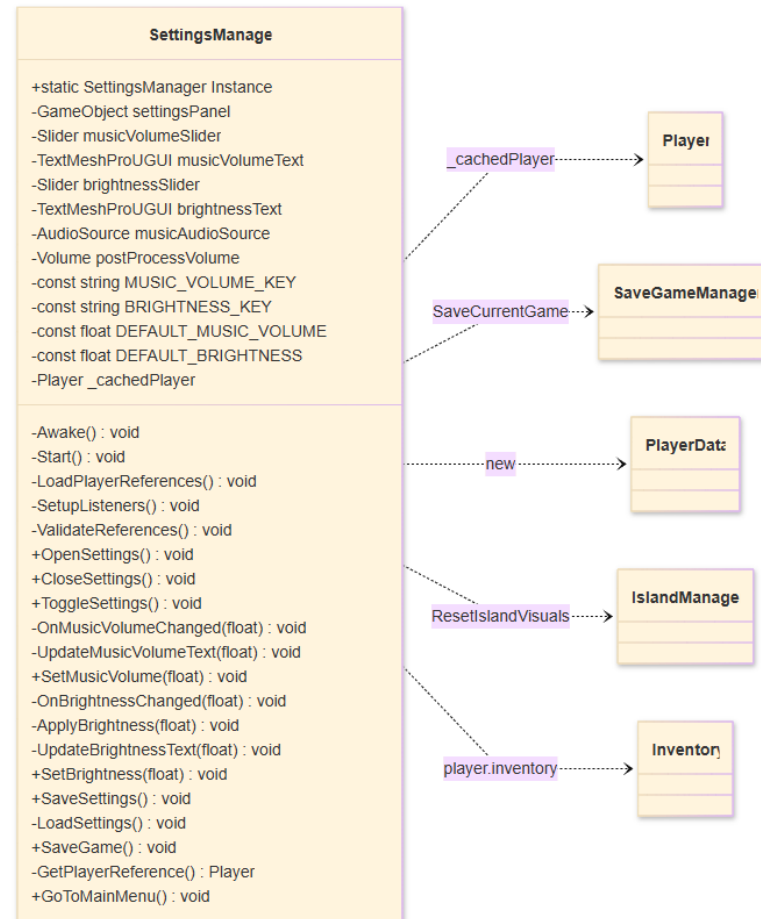


Diagrama de flujo

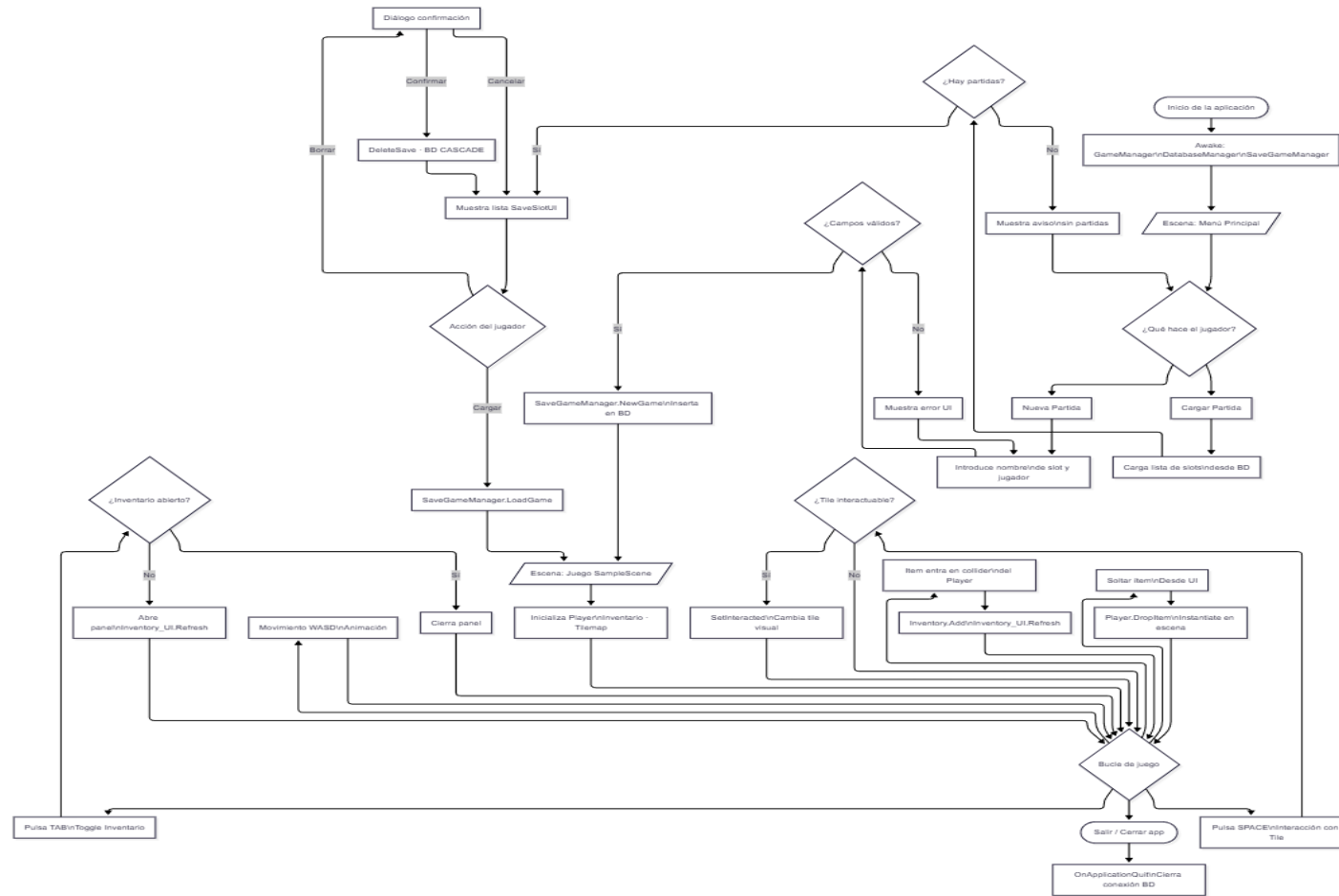
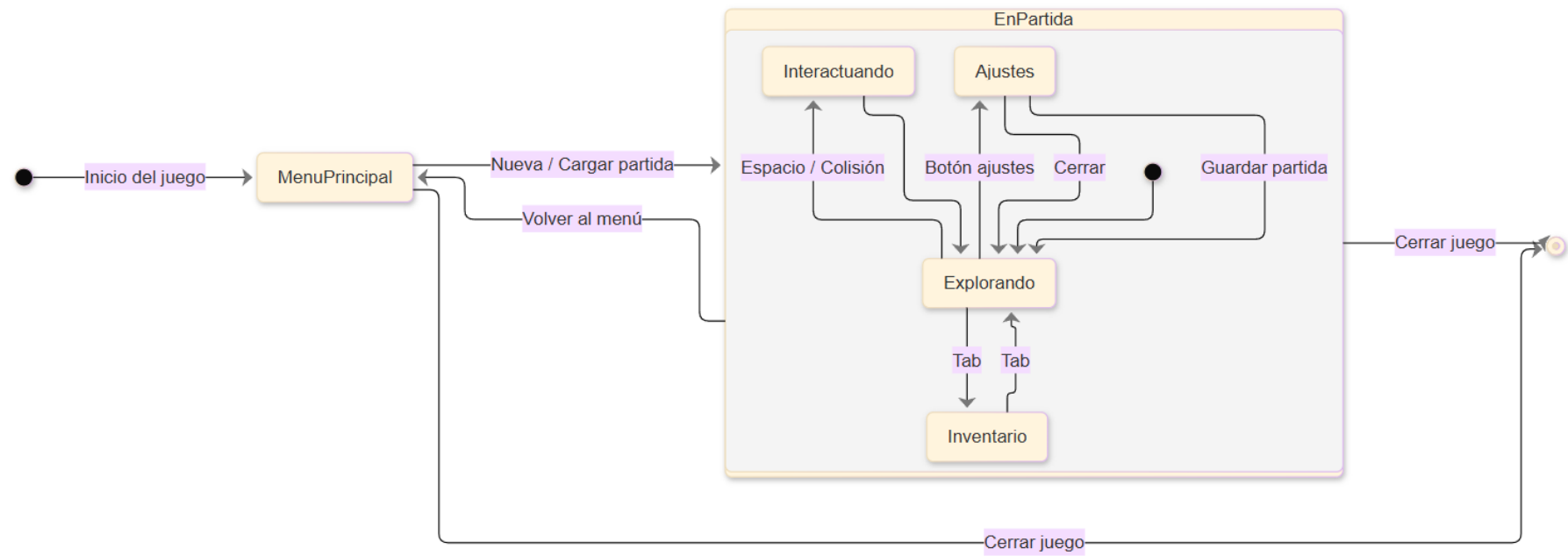
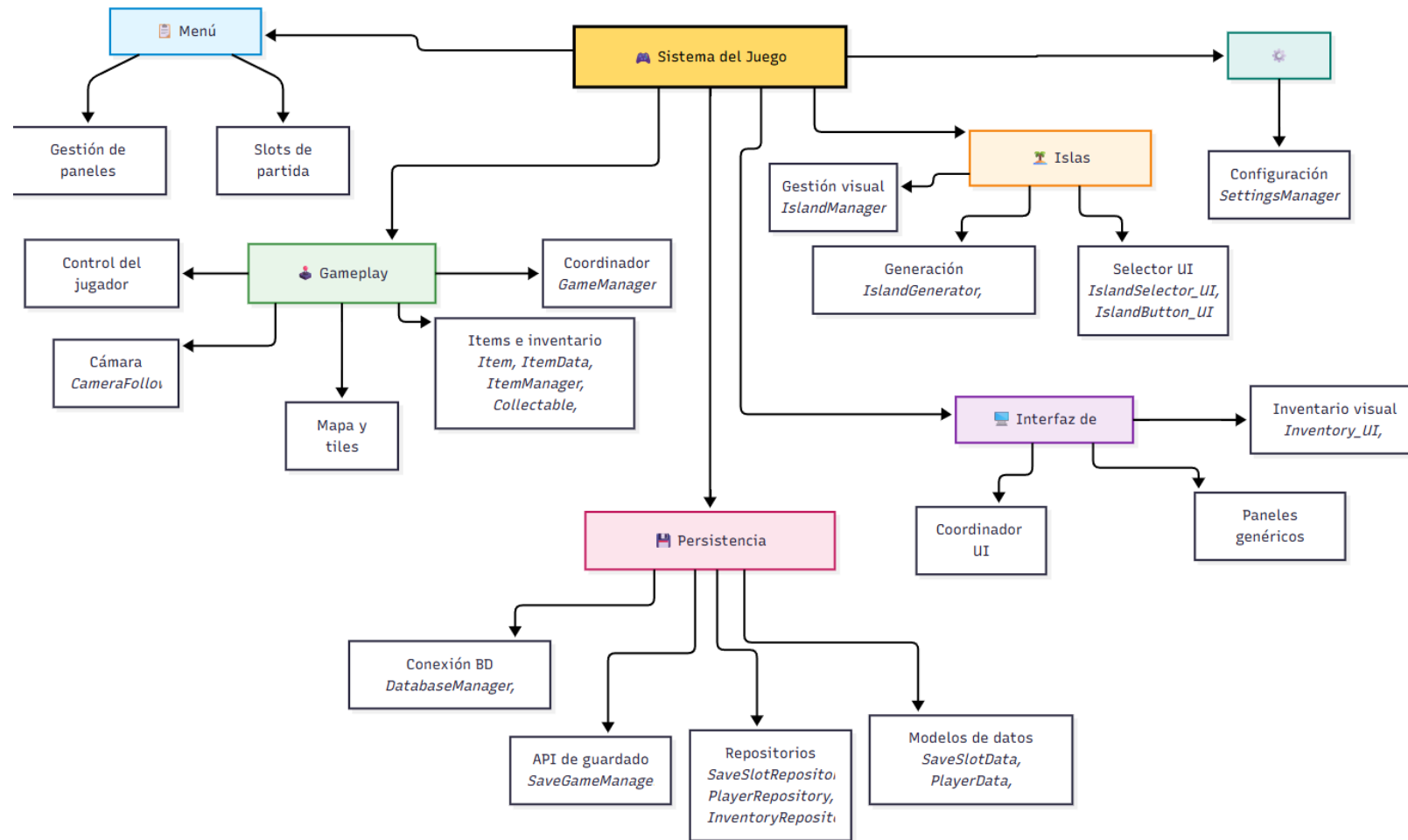


Diagrama de estados (Ciclo de vida)



Descomposición en módulos



Script de creación de la Base de datos

```
-- =====  
-- SCRIPT DE CREACIÓN DE TABLAS — savegame.db  
-- =====  
  
CREATE TABLE IF NOT EXISTS SaveSlots (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    slot_name TEXT NOT NULL,  
    player_name TEXT NOT NULL,  
    created_at TEXT NOT NULL,  
    last_saved TEXT NOT NULL,  
    play_time REAL DEFAULT 0,  
    island_seed INTEGER DEFAULT 0  
);  
  
CREATE TABLE IF NOT EXISTS Players (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    slot_id INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,  
    name    TEXT NOT NULL,  
    pos_x   REAL DEFAULT 0,  
    pos_y   REAL DEFAULT 0,  
    pos_z   REAL DEFAULT 0  
);  
  
CREATE TABLE IF NOT EXISTS Farms (  

```

```
id      INTEGER PRIMARY KEY AUTOINCREMENT,  
slot_id INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,  
farm_name TEXT NOT NULL,  
size_x  INTEGER DEFAULT 10,  
size_y  INTEGER DEFAULT 10,  
unlocked INTEGER DEFAULT 1  
);
```

```
CREATE TABLE IF NOT EXISTS FarmTiles (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    farm_id INTEGER NOT NULL REFERENCES Farms(id) ON DELETE CASCADE,  
    tile_x  INTEGER NOT NULL,  
    tile_y  INTEGER NOT NULL,  
    soil_state TEXT DEFAULT 'dry',  
    crop_id TEXT DEFAULT "",  
    growth_stage INTEGER DEFAULT 0,  
    days_planted INTEGER DEFAULT 0  
);
```

```
CREATE TABLE IF NOT EXISTS Inventory (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    slot_id INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,  
    item_id TEXT NOT NULL,  
    item_name TEXT NOT NULL,  
    quantity INTEGER DEFAULT 1,
```



```
slot_index INTEGER DEFAULT -1,  
item_data TEXT DEFAULT '{}'  
);
```